

第15章

存储过程与函数

15.1 MySQL存储过程与函数

OPTION

15.2 MySQL编程基础

OPTION

15.3 MySQL常用内置函数

OPTION

15.4 MySQL存储过程

OPTION

15.5 MySQL用户自定义函数

OPTION

15.6 游标的使用

OPTION



知识要点



应用实例

15.1 MySQL存储过程与函数

一、
OPTION

存储过程和函数概述

二、
OPTION

存储过程的优点

三、
OPTION

存储过程与函数的比较

存储过程和函数概述

1. 存储过程 (Stored Procedure)

是一组完成特定功能的SQL语言代码段，经编译后存储在数据库中，可被触发器、其他存储过程、程序设计语言所调用。

(1) 定义存储过程时，需指定其名称，用户名称并给出参数（如果该存储过程带有参数）调用执行指定的存储过程。

(2) 存储过程的功能由其过程体中的代码来决定，过程体“BEGIN...END”语句指定，其功能代码写在该语句范围内。

存储过程和函数概述

2. 存储函数 (Stored Function)

是由一组SQL语句和一些特殊的控制结构语句组成的代码片段，经过执行、运算后通过RETURN语句返回一个函数值。

- (1) 每个存储函数具有名称，用户按函数名并给出参数（如果有参数）调用执行指定的函数。
- (2) 存储函数的功能由其函数体中的代码来决定，函数体由“BEGIN...END”语句指定，其功能代码写在该语句范围内。

存储过程的优点

- (1) **增强了SQL语言的功能和灵活性。** 过程体可通过流程控制语句对SQL语句的执行进行流程控制，能完成较复杂的运算。
- (2) **便于被多次重复调用。** 存储过程经编译后被存储在其隶属的数据库中，可以被多次调用。
- (3) **能实现更快的执行速度。** 存储过程创建时，MySQL就对其编译、分析和优化，并且给出最终被存储在系统表中的执行计划；且首次执行后，存储在服务器的内存中，可多次执行。
- (4) **减少网络流量。** 调用存储过程时，网络中传送的只是调用语句，而不需要在网络中传送存储过程中的SQL语句代码。
- (5) **存储过程可作为一种安全机制来利用。** 只有受权的用户才具有对指定存储过程的使用权。

存储过程与函数的比较

存储过程	存储函数
参数可以有IN、OUT、INOUT三种类型	参数只有IN类型
需要用CALL语句调用存储过程，将存储过程作为一个独立的部分来执行	不需要CALL语句，可以直接调用存储函数，可以作为查询语句的一个部分来调用
过程体中不允许包含RETURN语句，不能有返回值，但可以通过OUT参数带回多个值	函数体中必须包含一条有效的RETURN语句，有且只有一个返回值，如：单个值或者表对象
存储过程可以调用存储函数	存储函数不能调用存储过程
主要用于执行并完成某个功能操作	主要用于计算并返回一个函数值

15.2 MySQL编程基础

OPTION

一、注释、定界符与语句块

OPTION

二、变量的定义和使用

OPTION

三、运算符与表达式

OPTION

四、流程控制语句



OPTION

注释、定界符与语句块

- 1. 注释
- 2. 定界符与DELIMITER命令
- 3. 语句块与BEGIN...END语句



注释、定界符与语句块

1. 注释

作用：

- ①对程序代码的功能及实现方式进行简要的解释和说明，便于对代码维护；
- ②可以把程序中暂时不用的语句加以注释，使它们暂时不被执行。



注释、定界符与语句块

1. 注释

三类注释符:

(1) `--`: 即双连线字符, 用于单行注释, 该符号到行尾的内容都为注释, 比如:

`-- 这是一行注释`

`USE teaching; -- 打开数据库`

(2) `#`: 即井号字符, 用于单行注释, 该符号到行尾的内容都为注释, 比如:

`SELECT * FROM s; #读出学生表s中的所有记录信息`

(3) `/*...*/`: `/*`注释的开头, `*/`注释的结尾, 所括起来的是注释, 用于多行注释。

`/* 程序代码的功能是:`

`打开数据库teaching*/`

OPTION 注释、定界符与语句块

2. 定界符与DELIMITER命令

(1) MySQL默认定界符 “;”

MySQL命令行客户端，命令行结束符默认是 “;”，即输入一行命令以 “;”结束，回车后，MySQL就会自动执行该语句行。

(2) DELIMITER命令格式：DELIMITER new_delimiter

其中：new_delimiter表示新定义的定界符，可以是//、\$\$等

功能：重新定义一个语句执行的结束符，用于客户端创建存储过程或函数（有多条语句）时，避免用户还未全部输入完其中的语句就执行，造成错误。

比如：DELIMITER \$\$表示新的定界符为 “\$\$”，自此开始，直到遇到下一个 “\$\$”，MySQL才会整体执行这段语句代码。



注释、定界符与语句块

3. 语句块与BEGIN...END语句

(1) 语句块

是由若干条语句构成的程序代码单元，在逻辑上被当作一个整体，语句块中的语句要么被执行，要么整体都不被执行。

(2) “BEGIN...END”语句

定义一个语句块，BEGIN和END为关键字，格式：

BEGIN

statement_list

END;

其中，statement_list表示是语句块，其中可包含若干条语句。

变量的定义和使用

- (1) MySQL编程可以使用变量，每个变量相当于一个容器，用于保存数据或用变量的值参加运算。
- (2) 变量具有**变量名**、**变量值**和**数据类型**三要素。
- ①**变量名**就是每个变量的名称，由用户给定，用于区分不同的变量；
 - ②**变量值**就是变量容器中的数据，变量值可以通过赋值进行改变；
 - ③**数据类型**是变量值所属的数据类型，可以是MySQL规定的类型，如**INT**、**VARCHAR**等。
- (3) 从变量的生存期和作用域范围上来看，MySQL编程中可以使用3种类型的变量，即：**用户会话变量**、**局部变量**、**系统变量**。

变量的定义和使用

OPTION

1、用户会话变量

MySQL数据库服务器可以被多个不同用户分别从多个客户端连接访问，用户连接成功后，服务器会在内存中为每一个用户连接开辟一个独立的会话连接空间，不同的会话空间互不干扰，连接断开后，则会话结束，会话空间将被释放。

- (1) **用户会话变量**是在某一用户会话连接空间内定义的变量，因此，用户会话变量隶属于特定会话。
- (2) **生命周期**是用户从客户端建立与MySQL服务器连接开始，到客户端与服务器连接断开的整个期间。

变量的定义和使用

1、用户会话变量

(3) **用户会话变量的定义**: 用户会话变量无需提前定义和赋值, 直接写明变量名 (前面加 “@”) 即可使用, 未赋初值时, 其初值为NULL。

(4) **用户会话变量的赋值**: 有两种方法, 一是SET语句赋值, 二是SELECT语句赋值。

①SET语句的格式:

`SET @variable_name1=expression1 [, @variable_name2=expression2,...];`

或者

`SET @variable_name1:=expression1 [, @variable_name2:=expression2,...];`

如: `SET @a=100,@b=200;` 或 `SET @a:=100,@b:=200;` #使用SET语句定义并初始化变量@a和@b

二、变量的定义和使用

OPTION

1、用户会话变量

②SELECT语句的格式:

```
SELECT @variable_name1:=expression1 [, @variable_name2:=expression2,...];
```

或者

```
SELECT expression1 INTO @variable_name1;
```

如: `SELECT @a:=100,@b:=200;` #使用SELECT语句定义并初始化变量@a和@b

```
SELECT 1 INTO @i; #使用SELECT语句定义并初始化变量@i
```

用户会话变量在定义时无需给出数据类型

变量的定义和使用

2、局部变量

局部变量是在MySQL程序代码中的一个语句块（BEGIN...END）内部定义的变量。

局部变量的作用范围仅限于定义该变量的语句块。

(1) 局部变量的定义

局部变量必须先用DECLARE命令声明后才可使用，格式为：

DECLARE **variable_name** [, ...] **datatype**(size) **DEFAULT** default_value

如：

DECLARE total_sale INT DEFAULT 0; #定义局部变量total_sale，类型为INT型，初值为0

DECLARE x, y INT DEFAULT 0; #同时定义x和y两个变量，类型都为INT型，初值都为0

DECLARE myname VARCHAR(10); #定义局部变量myname，类型为VARCHAR，长度为10

变量的定义和使用

OPTION

3、系统变量

(1) 系统变量的概念

系统变量由MySQL系统自动创建的变量，系统变量名前面有“@@”前缀。系统变量分为**全局变量** (GLOBAL) 与**会话变量** (SESSION)。

全局变量影响服务器整体操作，是在MySQL启动的时候由服务器自动将它们初始化为默认值。

会话变量影响其各个客户端连接的操作，是在MySQL服务器每次针对一个客户端建立一个新的连接的时候，创建的系统变量，由MySQL来初始化，将当前所有全局变量的值复制一份，作为会话变量。

变量的定义和使用

3、系统变量

(2) 查看系统变量名称及取值

使用`SHOW VARIABLES`语句查看系统变量的名称以及取值情况。有以下几种常用方法：

- ①查询所有的全局变量，语句：`SHOW GLOBAL VARIABLES;`
- ②查询所有的会话变量，语句：`SHOW SESSION VARIABLES;`或`SHOW VARIABLES;`
- ③查询某一个系统变量的取值，语句如下：

`SELECT @@GLOBAL.var_name;` #查询var_name指定的某个全局变量

`SELECT @@SESSION.var_name;` #查询var_name指定的某个会话变量

`SELECT @@var_name;` #查询由var_name指定的某个变量，优先查询会话变量，再查询全局变量

比如，`SELECT @@version;` #查看全局变量@@version的值，可查看MySQL的版本



运算符与表达式

1. 算术运算符与算术表达式
2. 比较运算符与关系表达式
3. 逻辑运算符与逻辑表达式
4. 位运算符及其表达式
5. 运算符的优先级



运算符与表达式

1. 算术运算符与算术表达式

算术运算符	含义	用法	算术表达式示例
+	加	$a+b$	10+20的值为30
-	减	$a-b$	10-20的值为-10
*	乘	$a*b$	2*3的值为6
/	实数除, 商为实数	a/b	2/4的值为0.5
DIV	整数除, 商为整数	$a \text{ DIV } b$	2 DIV 4的值为0
%或MOD	求余数	$a \% b$ 或 $a \text{ MOD } b$	5%3的值为2

由算术运算符、括号将运算对象连接起来形成的运算式子称为算术表达式。



运算符与表达式

2. 比较运算符与关系表达式

比较运算符	含义	用法	关系表达式示例及结果
=	相等	a=b	2=5的值为0
>	大于	a>b	5>-2的值为1
<	小于	a<b	2<-9的值为0
>=	大于或等于	a>=b	5>=3的值为1
<=	小于或等于	a<=b	10<=5的值为0
<> 或 !=	不等于	a<>b或a!=b	-5<>3的值为1
<=>	安全的等于，不会返回NULL	a<=>b	1<=>NULL的值为0
IS NULL	判断一个值是否为NULL	a IS NULL	NULL IS NULL的值为1
IS NOT NULL	判断一个值是否不为NULL	a IS NOT NULL	-3 IS NOT NULL的值为1
BETWEEN AND	判断一个值是否落在两个值之间	a BETWEEN b and c	10 BETWEEN 1 AND 20的值为1
IN	判断一个值是否处在一个集合中	a IN (集合)	5 IN (10,20,30,40) 的值为0
NOT IN	判断一个值不在一个集合中	a NOT IN (集合)	5 NOT IN (10,20,30,40) 的值为1
LIKE	模式匹配	a LIKE p	'BEI' LIKE 'BEI%'的值为1
NOT LIKE	模式不匹配	a NOT LIKE p	'BEI' NOT LIKE 'BEI%'的值为0
REGEXP	正则表达式匹配	a REGEXP reg	's1' REGEXP '^s[1-9]'的值为1

由关系运算符、括号将运算对象连接起来形成的运算式子称为关系表达式。



运算符与表达式

2. 比较运算符与关系表达式

说明:

(1) 关系表达式的运算结果可以为1（真）、0（假）或NULL。当表达式成立时，结果为1；不成立时，结果为0；当任意参加运算一方为NULL，则结果为NULL。

(2) “IN”判断一个数据是否在集合中，集合可以是一个SELECT语句查询结果集，也可直接给出，(元素1, 元素2, ...)，用法是：**a IN (元素1, 元素2, ...)**，若a与集合中某个元素相等，则结果为1，否则为0。

(3) “LIKE”运算符，用法是：**a LIKE p**，a表示字符串，p表示模式，用来判断一个字符串a是否匹配一个模式字符串p，若匹配，则结果为1，否则，结果为0；若运算对象其中之一为NULL，结果为NULL。

在模式字符串中可以使用通配符“%”和“_”，“%”可代替任意多个字符，“_”表示任意一个字符。

比如，'张玉涛' LIKE '张%'的结果为1。

运算符与表达式

2. 比较运算符与关系表达式

(4) “REGEXP”运算符，用法：**a REGEXP p**，a表示字符串，p表示用正则表达式表示的模式字符串，用来判断字符串a是否匹配模式字符串p，如果匹配，则结果为1，否则为0。

在正式表达式中，**p中常用的通配符有：**

- ① “^”匹配以该字符后面的字符开头的字符串。
- ② “\$”匹配以该字符前面的字符结尾的字符串。
- ③ “.”匹配任意一个字符。
- ④ “[...]”匹配在方括号中的任何字符。如：“[abc]”表示匹配字符a、b或c。可以使用“-”表示一个范围，如：“[a-z]”表示匹配字符a至z中的任意字符，“[0-9]”表示匹配任意一个数字字符。

运算符与表达式

2. 比较运算符与关系表达式

⑤ **"*"** 匹配0个或多个在它前面的字符。

如, "a*"表示匹配任意多个字符a, "[0-9]*"匹配任意多个数字字符。

再如, 'abc' REGEXP '[a-z]*'的运算结果为1。

⑥ **"{n, m}"**匹配前面的字符串至少n次, 至多m次。

比如, 'aa' REGEXP 'a{2,5}'的值为1, 模式字符串 "a{2,5}"表示由2-5个字符 "a"组成的字符串。

运算符与表达式

3. 逻辑运算符与逻辑表达式

逻辑运算符	含义	用法	逻辑表达式示例及结果
NOT或!	逻辑非，单目运算符。若参加运算的操作数为非零，则运算结果为0；若操作数为0，则运算结果为1；若操作数为NULL，则运算结果为NULL	NOT a或!a	!5的值为0
AND 或&&	逻辑与，若参加运算的两个操作数的值都为非零（且不为NULL），则运算结果就为1；若有任何一个操作数为0，则运算结果为0；若一个操作数为NULL且另一操作数为非零或NULL，则运算结果为NULL	a AND b	5 AND NULL的值为NULL
OR或	逻辑或，若参加运算的两个操作数的值任何之一为非零，则运算结果就为1；若两个操作数都为0，则运算结果为0；若存在一个操作数为0或NULL，另一操作数为NULL，则运算结果为NULL	a OR b	0 OR 5的值为1
XOR	逻辑异或，当任意一个操作数为NULL，运算结果为NULL。对于非NULL的操作数，若两个操作数都是非零或都为0，则结果为0；若一个为0，另一个为非零，则运算结果为1	a XOR b	5 XOR 1的值为0

运算符与表达式

3. 逻辑运算符与逻辑表达式

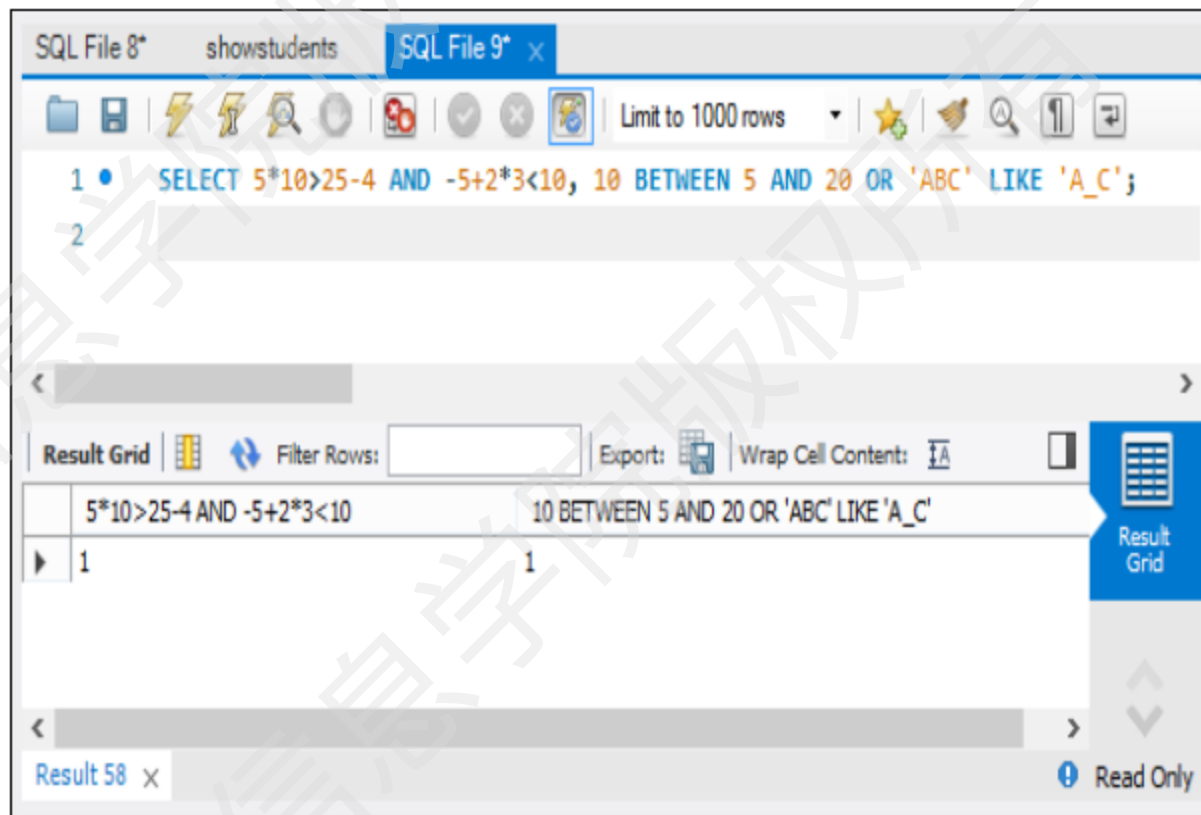
用逻辑运算符、括号将运算对象连接起来形成的运算式子叫逻辑表达式，其运算结果为1（真）、0（假）或NULL。

编程中常用逻辑表达式表示复杂的条件。

比如，

$5*10>25-4$ AND $-5+2*3<10$ 的值都为1；

10 BETWEEN 5 AND 20 OR 'ABC' LIKE 'A_C' 的值都为1。





运算符与表达式

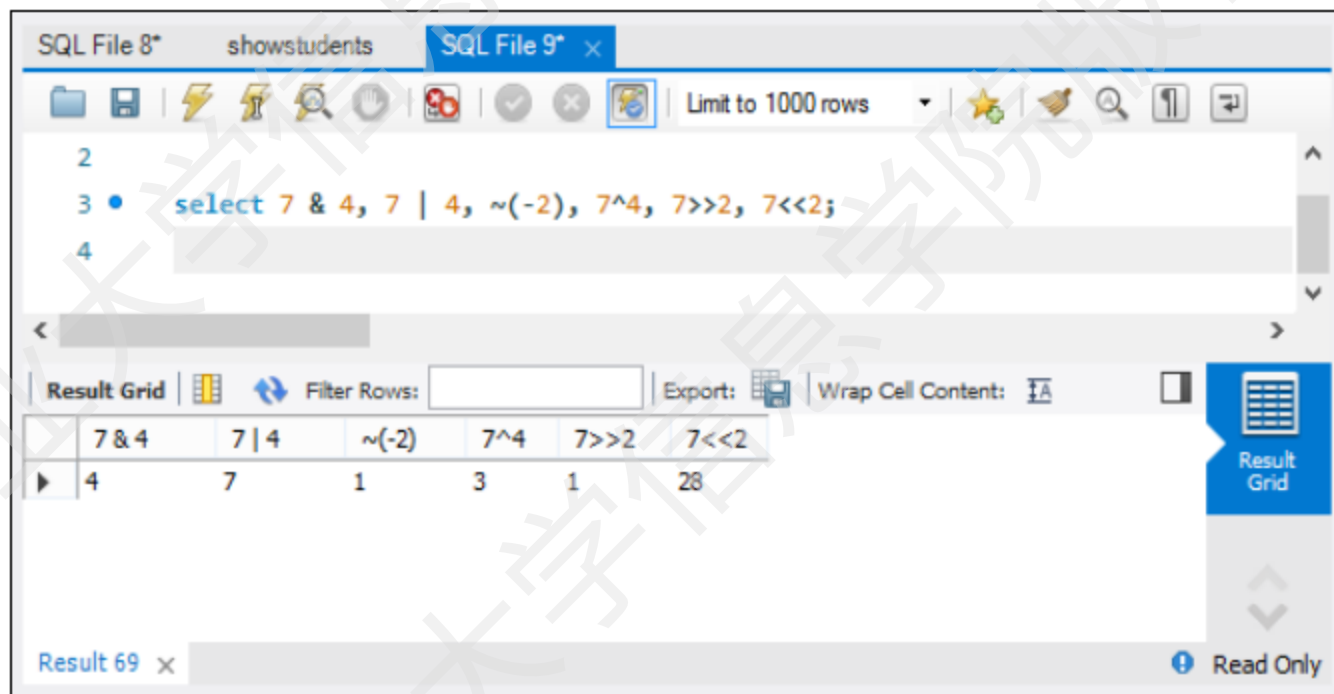
4. 位运算符及其表达式

位运算符是对操作数的二进制数上的二进制位进行运算的运算符。

比如，表达式 $7 \& 4$ 的值为4。7对应的二进制数为00000111，4对应的二进制数为00000100。

```

00000111<
&<
00000100<
←
00000100<
    
```





运算符与表达式

5. 运算符的优先级

优先级（从高到低）	运算符	说明
1	()	小括号
2	!	逻辑非
3	+, -, ~	正、负、按位取反
4	^	按位异或
5	*, /, DIV, %或MOD	乘、除、整数除、求余数
6	+, -	加、减
7	<<, >>	按位左移、按位右移
8	&	按位与运算
9		按位或运算
10	=, <=>, <, <=, >, >=, <>, !=, IN, IS NULL, LIKE, REGEXP	各种比较运算符
11	BETWEEN...AND	比较运算符
12	NOT	逻辑非
13	AND或&&	逻辑与
14	XOR	逻辑异或
15	OR或	逻辑运算符
16	=（赋值号）、:=	赋值运算符



运算符与表达式

5. 运算符的优先级

表达式中有多个运算符时，高优先级的运算符先于低优先级的运算符进行计算；具有相同优先级的运算符，则一般按照从左到右的方向进行运算。

比如，

表达式 $(2021 \text{ MOD } 4)=0 \text{ AND } (2021 \text{ MOD } 100 \neq 0)$ 的值为0。

表达式 `'陈佳' LIKE '陈%' AND 19 BETWEEN 17 AND 21 AND 89 >= 85` 的值为1。

表达式 $1 \parallel 4 \text{ XOR } 4$ 的值为1。

表达式 $1 < < 2 + 8 - 3 * 9 \text{ MOD } 4^3$ 的值为16。

四、
OPTION

流程控制语句

1. IF语句
2. CASE语句
3. WHILE语句
4. REPEAT语句
5. LOOP语句
6. LEAVE语句和ITERATE语句

四、

OPTION

流程控制语句

1. IF语句

IF语句用来进行条件判断，根据是否满足条件来执行不同的语句，语句的格式如下：

```
IF condition1 THEN
    statement_list1
[ELSEIF condition2 THEN
    statement_list2 ]
...
[ELSE
    statement_listn]
END IF;
```

四、

OPTION

流程控制语句

1. IF语句

【例15-1】从教学数据库teaching中的选课表sc中求出学号为s1同学的平均成绩，如果此平均成绩大于或等于60分，则输出“Pass!”信息。

```
BEGIN
```

```
    IF (SELECT AVG(score) FROM sc WHERE sno='s1')>=60 THEN
```

```
        SET @infor='Pass!';
```

```
    ELSE
```

```
        SET @infor='Fail!';
```

```
    END IF;
```

```
    SELECT @infor;
```

```
END
```

四、

OPTION

流程控制语句

2. CASE语句

CASE语句是一种分多支语句结构，有两种格式：

(1) CASE语句的格式1

CASE expression

WHEN value1 THEN statement_list1

[WHEN value2 THEN statement_list2]

...

[ELSE statement_listn]

END

【例15-2】从数据库教学数据库teaching中的学生表s选取sno和sex，如果sex字段值为“男”，则输出“M”；如果为“女”则输出“F”。

```
SELECT sno,
       CASE sex
         WHEN '男' THEN 'M'
         WHEN '女' THEN 'F'
       END
       AS sex
FROM s;
```



流程控制语句

2. CASE语句

CASE语句是一种分多支语句结构，该语句有两种格式：

(2) CASE语句的格式2

CASE

WHEN search_condition1 THEN statement_list1

[WHEN search_condition2 THEN statement_list2]

...

[ELSE statement_listn]

END

【例15-3】从教学数据库teaching中的sc表查询所有同学选课的成绩情况，凡成绩为空的输出“未考”，小于60分的输出“不及格”，60分至70分的输出“及格”，70分至90分的输出“良好”，大于或等于90分的输出“优秀”。

四、

OPTION

流程控制语句

3. WHILE语句

WHILE语句可以实现循环结构，格式如下：

```
[begin_label:]WHILE condition DO
```

```
    statement_lists
```

```
END WHILE [begin_label];
```

【例15-4】编写程序，求 $1+2+3+\dots+100$ 的值。

四、

OPTION

流程控制语句

4. REPEAT语句

REPEAT语句的格式如下：

[begin_label:]REPEAT

statement_lists

UNTIL condition

END REPEAT [begin_label];

【例15-6】 求解满足 $1^2+2^2+3^2+\dots+i^2<1000$ 的*i*的最大值。



流程控制语句

5. LOOP语句

LOOP语句也是用于构成循环的语句，但是它构成的是一个无条件的循环，语句的格式如下：

```
[begin_label:]LOOP
```

```
statement_lists
```

```
END LOOP [begin_label];
```



流程控制语句

6. LEAVE语句和ITERATE语句

(1) LEAVE语句

LEAVE语句是从循环体中跳出循环的语句，语句格式：

LEAVE label ;

(2) ITERATE语句

ITERATE语句的作用是用在循环体中，中止当前执行的本次循环，直接进入下一次循环。格式：

ITERATE label;

15.3 MySQL 常用内置函数

一、
OPTION

数学函数

二、
OPTION

字符串函数

三、
OPTION

日期和时间函数

四、
OPTION

系统信息函数

五、
OPTION

聚合与统计函数

六、
OPTION

加密函数

七、
OPTION

其他函数

数学函数

OPTION

函数类别	函数名	功能	示例
三角函数	SIN(x)	返回以弧度表示的x的正弦	SIN(0)的值为0
	COS(x)	返回以弧度表示的x的余弦	COS(0)的值为1
	TAN(x)	返回以弧度表示的x的正切	TAN(3.14159/4) 的值为0.999998
	COT(x)	返回以弧度表示的x的余切	COT(1)的值为0.642092
反三角函数	ASIN(x)	返回以x为正弦值的角（弧度）	ASIN(0)的值为0
	ACOS(x)	返回以x为余弦值的角（弧度）	ACOS(1)的值为0
	ATAN(x)	返回以x为正切值的角（弧度）	ATAN(0)的值为0
角度弧度转换	DEGREES(x)	把x从弧度转换为角度	DEGREES(1)的值为57.29°
	RADIANS(x)	把x从角度转换为弧度	RADIANS(90.0)的值为1.570796
指数函数	EXP(x)	返回以e为底的x次幂	EXP(1)的值为2.718282
	POWER(x,y)	返回以x为底的y次幂	POWER(2,4)的值为16
对数函数	LOG(x)	返回x的以e为底的自然对数值	LOG(1)的值为0
	LOG10(x)	返回x的以10为底的常用对数值	LOG10(10)的值为1
平方根函数	SQRT(x)	返回x的平方根	SQRT(1)的值为1
取近似值函数	CEILING(x)	返回大于等于x的最小整数	CEILING(-5.6)的值为-5
	FLOOR(x)	返回小于等于x的最大整数	FLOOR(-5.2) 的值为-6
	ROUND(x,n)	将x四舍五入为指定的小数位n	ROUND(5.6782,2) 的值为5.68
	TRUNCATE(x,n)	保留x的n位小数并返回	TRUNCATE(12.567,2)的值为12.56
符号函数	ABS(x)	返回x的绝对值	ABS(-3.4) 的值为3.4
	SIGN(x)	测试x的正负号，返回0、1或-1	SIGN(-3.4) 的值为-1
圆周率函数	PI()	返回圆周率	PI()的值为3.141593
随机数函数	RAND()	返回(0,1)之间的随机数	

字符串函数

OPTION

函数名	功能	示例
ASCII(s)	返回字符串s的第一个字符的ASCII码	ASCII('abc')的值为97
CHAR_LENGTH(s) 或CHARACTER_LENGTH(s)	返回字符串s的字符数（长度）	CHAR_LENGTH('北京')的值为2
CONCAT(s1,s2...sn)	字符串s1, s2, ..., sn等多个字符串合并为一个字符串	CONCAT('首都','北京')的值为'首都北京'
FIND_IN_SET(s1,s2)	分析逗号分隔的字符串s2, 如果在s2中存在s1, 则返回s1在s2中出现的位置	FIND_IN_SET('北京','首都,北京,大学,林业')的值为2
FORMAT(x,n)	函数可以将数字x进行格式化"#,###.##", 将x保留到小数点后n位, 最后一位四舍五入。	FORMAT(315500.7489, 2)的值为315,500.75
LOCATE(s1,s)或LOCATE(s1,s,pos)	从字符串s中返回s1首次出现的位置（从1开始计数）, 如果给定参数pos, 则从pos指定的位置开始查找。未找到, 返回0	LOCATE('ab','xyzabcefabc')的值为4
INSTR(s1,s)	从字符串s1中返回s首次出现的位置（从1开始计数）, 未找到时返回0	INSTR('xyzabcefabc','ab')的值为4
LEFT(s,n)	从字符串s的左端截取n个字符形成子串	LEFT('首都北京',2)的值为'首都'
LOWER(s)或LCASE(s)	将字符串s的所有字母变成小写字母	LCASE('Beijing')的值为'beijing'
LTRIM(s)	去掉字符串s开始处（左端）的空格	RTRIM(' CAPITAL ')的值为'CAPITAL '
POSITION(s1 IN s)	从字符串s中返回s1首次出现的位置	POSITION('ab' IN 'xyzabcefabc')的值为4
REPEAT(s,n)	将字符串s重复n次, 形成新的字符串	REPEAT('北京',2)的值为'北京北京'
REPLACE(s,s1,s2)	将字符串s2替代字符串s中的子串s1	REPLACE('abc','a','x')的值为'xbc'
REVERSE(s)	将字符串s的顺序反过来	REVERSE('abc')的值为'cba'
RIGHT(s,n)	从字符串s的右端截取n个字符组成子串	RIGHT('BEIJING',4)的值为'JING'
RTRIM(s)	去掉字符串s结尾处（右端）的空格	RTRIM(' CAPITAL ')的值为' CAPITAL'
SPACE(n)	返回n个空格所组成的字符串	SPACE(10)返回10个空格
STRCMP(s1,s2)	比较字符串s1和s2, 如果s1与s2相等返回0, 如果s1>s2返回1, 如果s1<s2返回-1	STRCMP('bei','BEIJING')的值为-1
SUBSTRING(s, start, length) 或SUBSTR(s, start, length) 或MID(s,n,len)	从字符串s的start位置截取长度为length的子字符串	SUBSTRING('BEIJING',1,3)的值为'BEI' SUBSTR('BEIJING',1,3)的值为'BEI' MID('BEIJING',1,3)的值为'BEI'
TRIM(s)	去掉字符串s开始和结尾处的空格	TRIM(' CAPITAL ')的值为'CAPITAL'
UCASE(s)或UPPER(s)	将字符串s转换为大写	UCASE('beijing')的值为'BEIJING'

日期和时间函数

函数名	功能	示例
CURDATE()或CURRENT_DATE()	返回当前日期	CURDATE()的值为'2021-2-6'
CURTIME()或CURRENT_TIME()	返回当前时间	CURTIME()的值为'15:30:00'
DATE_ADD(date,interval exp type)	返回日期date加上间隔exp的结果(但exp必须按关键字进行格式化)	DATE_ADD("2020-12-31 23:59:59",INTERVAL 1 SECOND)的值为'2021-01-01 00:00:00'
DATE_FORMAT(date,f)	依据指定的格式对日期date进行格式化	DATE_FORMAT(NOW(),'%b %d %Y %h:%i %p')的值为'Feb 06 2021 03:00 PM'
DATE_SUB(date,interval exp type)	返回日期date减去间隔exp的结果(但exp必须按关键字进行格式化)	SELECT DATE_SUB("2021-1-1 00:00:00",INTERVAL 1 SECOND)的值为'2020-12-31 23:59:59'
DATEDIFF(date1,date2)	返回date1与date2两个日期期间的天数	DATEDIFF(CURDATE(),'2021-1-1')的值为36
DAY(date)或DAYOFMONTH(date)	返回日期date是该月的第几天(1-31)	DAYOFMONTH(CURDATE())的值为6
DAYNAME(date)	返回日期date是星期几(英文)	DAYNAME(CURDATE())的值为'Saturday'
DAYOFWEEK(date)	返回日期date星期索引	DAYOFWEEK(CURDATE())的值为7
DAYOFYEAR(date)	返回日期date一年中的天(1-366)	DAYOFYEAR(CURDATE())的值为37
HOUR(time)	返回时间time的小时部分	HOUR(CURTIME())的值为15
MINUTE(time)	返回时间time的分钟值(0-59)	MINUTE('15:35:40')的值为35
MONTH(date)	返回日期date的月份值(1-12)	MONTH('2021-2-6')的值为2
MONTHNAME(date)	返回日期date的月份名称	MONTHNAME('2021-2-6')的值为'February'
NOW()	返回当前日期和时间	NOW()的值为'2021-02-06 15:35:00'
QUARTER(date)	返回日期date所属季度(1-4)	QUARTER(CURDATE())的值为1
SECOND(time)	返回时间time的秒数(0-59)	SECOND('15:35:40')的值为40
SYSDATE()	返回函数执行时的时间	同NOW()函数
TIME_FORMAT(time,f)	对时间time用格式f进行格式化	TIME_FORMAT('15:30:00','%l:%i %p')的值为'3:30 PM'
UTC_DATE()	返回当前UTC日期	UTC_DATE()的值为'2021-02-06'
UTC_TIME()	返回当前UTC时间	UTC_TIME()的值为'07:35:00'
WEEK(date)或WEEKDAY(date)或WEEKOFYEAR(date)	返回日期date的日历周(1-53)	WEEKOFYEAR('2021-2-6')的值为5
YEAR(date)	返回日期date的年份	YEAR('2021-2-6')的值为2021



日期和时间函数

在使用日期和时间的格式化函数DATE_FORMAT(date,f)以及TIME_FORMAT(time,f)函数时，需要用格式f对date或time进行格式化，常用的格式化符号代码及其功能如下表：

格式符	功能
%a	缩写的星期名 (Sun...Sat)
%b	月份的英文缩写名称 (Jan...Dec)
%c	月份，数字形式 (1...12)
%d	月份中的天数 (0...31)
%H	小时 (00...23)
%h	小时 (01...12)
%I	分钟 (00...59)
%i	分钟 (00...59)
%j	一年中的天数 (001...366)
%l	小时 (1...12)
%M	月份英文名称 (January...December)
%m	月份，数字形式 (00...12)
%p	上午 (AM) 或下午 (PM)
%r	12小时制时间 (小时hh:分钟mm:秒数ss后加AM或PM)
%S	秒 (00...59)
%T	时间，24小时制 (小时hh:分钟mm:秒数ss)
%W	英文星期几名称 (Sunday...Saturday)
%w	一周中的天数式 (0=周日...6=周六)
%Y	年份，数字形式，4位数
%y	年份，数字形式 (2位数)

比如，

SELECT TIME_FORMAT('17:20:35','%r');的结果为'05:20:35 PM'。

SELECT DATE_FORMAT('21-2-6','%M/%d/%Y');的结果为'February/06/2021'。

SELECT DATE_FORMAT('21-2-6','%M/%d/%Y %W');的结果为
'February/06/2021 Saturday'。

SELECT DATE_FORMAT('2021-02-06 11:07:45','%M %D,%Y');的结果为
'February 6th,2021'。

SELECT DATE_FORMAT('2021-02-06','%d %b %Y');的结果为'06 Feb 2021'。

四、

OPTION

系统信息函数

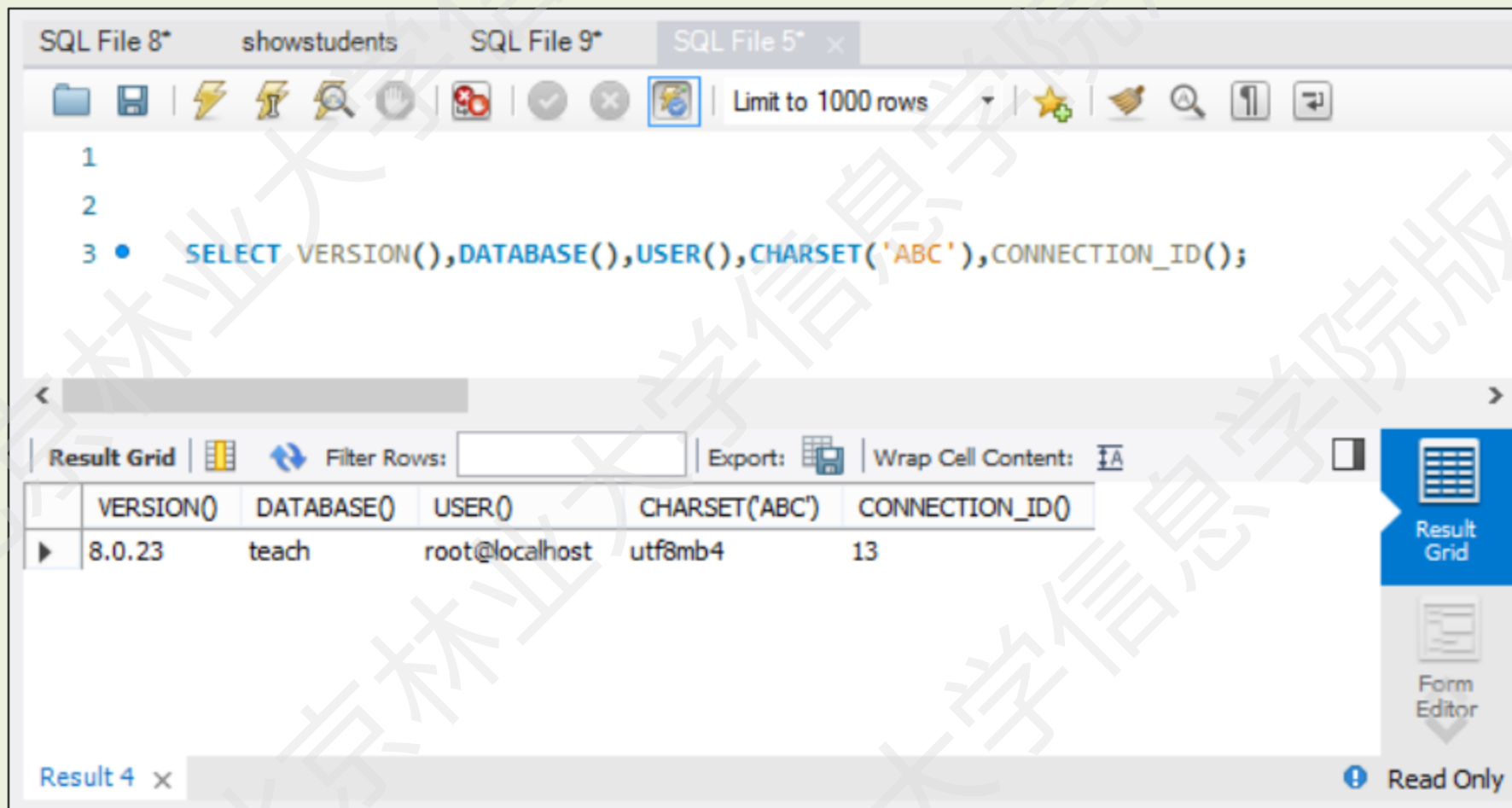
函数名	功能
CHARSET(str)	返回字符串str的字符集
CONNECTION_ID()	返回当前用户的连接ID
CURRENT_USER()	返回当前用户名称
DATABASE()	返回当前数据库名
FOUND_ROWS()	返回最后一个SELECT查询进行检索的总行数
LAST_INSERT_ID()	返回系统自动产生的最后一个auto_increment（自动增长）的值
SCHEMA()	返回当前数据库名
SESSION_USER()	返回当前用户名称
USER()或SYSTEM_USER()	返回当前用户名称
VERSION()	返回数据库的版本号

四、

OPTION

系统信息函数

如：显示MySQL的版本号、当前连接的数据库名、用户名、字符集、连接ID



The screenshot shows a MySQL IDE window with a SQL query editor and a results grid. The query is:

```
SELECT VERSION(),DATABASE(),USER(),CHARSET('ABC'),CONNECTION_ID();
```

The results grid displays the following data:

VERSION()	DATABASE()	USER()	CHARSET('ABC')	CONNECTION_ID()
8.0.23	teach	root@localhost	utf8mb4	13

The IDE interface includes a toolbar with various icons, a 'Limit to 1000 rows' dropdown, and a 'Result Grid' button on the right. The status bar at the bottom indicates 'Result 4' and 'Read Only'.

五、
OPTION

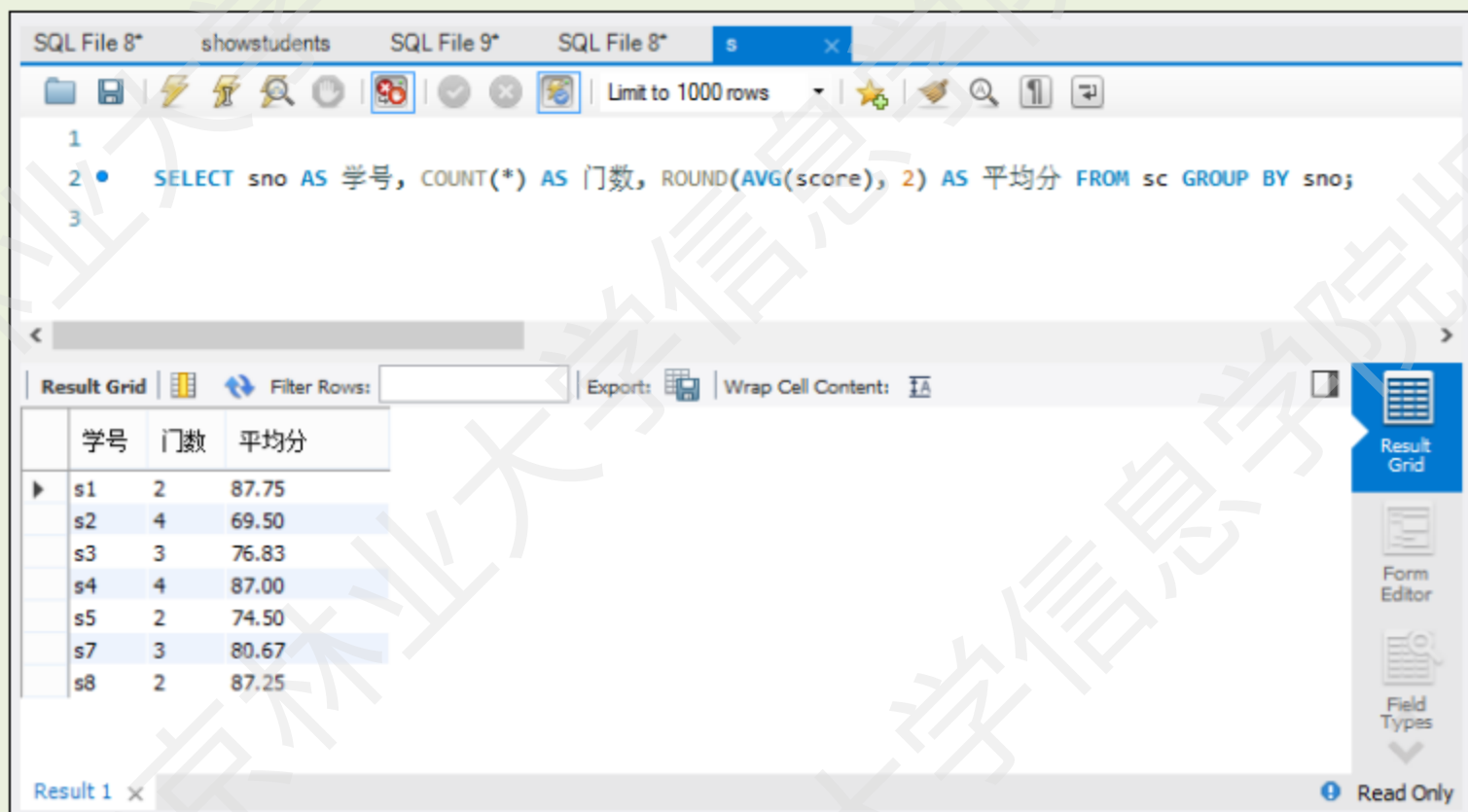
聚合与统计函数

函数名	用法与功能
AVG(col)	对列col求平均值，只计算非NULL值的平均值，即不包括NULL值，但包括0值
COUNT({[ALL DISTINCT] col} *)	COUNT(*)统计查询结果集中的总记录行数，无论是否包含NULL值； COUNT(col)或COUNT(ALL col)以列col的值统计查询结果集中的个数（行数），不包括NULL值，但包括重复值； COUNT(DISTINCT col)以列col的值统计查询结果集中的个数（行数），不包括NULL值且去掉重复值
MAX(col)	对列col求最大值，不包括NULL值
MIN(col)	对列col求最小值，不包括NULL值
SUM(col)	对列col求总和，不包括NULL值。只适用于对数值类字段求总和，比如字段类型可以是INT、FLOAT、DOUBLE、DECIMAL等类型
STD(col)或STDDEV(col)	对列col的值求标准差
VARIANCE(col)	对列col的值求方差

五、 OPTION

聚合与统计函数

【例15-8】从教学数据库teaching的选课表sc中统计每一位已选课同学的选课门数以及平均成绩，要求显示学号、门数和平均分。



The screenshot shows a MySQL IDE window with a SQL query and its results. The query is:

```
SELECT sno AS 学号, COUNT(*) AS 门数, ROUND(AVG(score), 2) AS 平均分 FROM sc GROUP BY sno;
```

The results are displayed in a table with the following columns: 学号 (Student ID), 门数 (Number of Courses), and 平均分 (Average Score). The results are as follows:

学号	门数	平均分
s1	2	87.75
s2	4	69.50
s3	3	76.83
s4	4	87.00
s5	2	74.50
s7	3	80.67
s8	2	87.25



加密函数

函数名	功能
MD5(str)	对字符串str使用信息摘要算法进行加密，加密后形成长度32位的密文，该加密算法不可逆
SHA(str)或SHA1(str)	对字符串str使用安全散列算法进行加密，加密后形成长度40位的密文，该加密算法不可逆
AES_ENCRYPT(str,key)	返回用密钥key对字符串str利用高级加密标准算法（AES）加密后的结果，结果是一个二进制字符串，以BLOB类型存储
AES_DECRYPT(crypt_str,key)	返回用密钥key对使用AES_ENCRYPT()函数加密后的密文crypt_str进行解密的结果
COMPRESS(string_to_compress)	该函数对字符串string_to_compress进行压缩，且返回一个二进制串。压缩后的字符串可以通过UNCOMPRESS()函数来解压缩
UNCOMPRESS(string_to_uncompress)	该函数解压缩一个通过COMPRESS()函数压缩的字符串string_to_uncompress

1. 控制流函数

函数名	功能	示例
IF(expr,v1,v2)	如果表达式expr的值为真，则返回v1的值，否则返回v2的值	IF(50<100, 5, 10)的值为5
IFNULL(v1,v2)	如果表达式expr的值不是NULL，则返回v1，否则返回v2	IFNULL(1/0, 4)的值为4

2. 数据类型转换函数

函数名	功能	示例
CAST(value AS type)	将value的数据类型转换为type类型	CAST(3.14 AS CHAR(4))的值为字符串'3.14'
CONVERT(value, type)	把value的数据类型转换为type类型	CONVERT('3.14',FLOAT)的值为3.14（数值）

15.4 MySQL 存储过程

一、
OPTION

创建存储过程

二、
OPTION

调用存储过程

三、
OPTION

查看存储过程

四、
OPTION

修改存储过程

五、
OPTION

删除存储过程

六、
OPTION

存储过程的参数

创建存储过程

1. 在代码窗口中使用CREATE PROCEDURE语句创建存储过程
2. 利用可视化管理工具MySQL Workbench功能菜单创建存储过程

创建存储过程

1. 在代码窗口中使用CREATE PROCEDURE语句创建存储过程

CREAE PROCEDURE语句的语法格式如下：

```
CREATE [DEFINER = { user | current_user }] PROCEDURE
```

```
    procedure_name([procedure_parameter[, ...]])
```

```
    [characteristic ...]
```

```
BEGIN
```

```
    routine_body
```

```
END
```

重要说明：

(1) **procedure_parameter**表示存储过程的参数（即形参），可选。每个参数由3部分组成，即参数传递类型、名称和数据类型，形式：

[IN | OUT | INOUT] parameter_name type

① 参数传递类型有**IN**、**OUT**、**INOUT**三种类型。**IN**输入类型；**OUT**输出类型；**INOUT**既可是输入类型，也可是输出类型。省略默认为**IN**。

② **parameter_name**表示是参数的名称，须由用户给出。

③ **type**表示参数的数据类型，可以是MySQL所支持的所有数据类型。

当前用户必须有CREATE PROCEDURE的权限。

创建存储过程

说明:

(2) `characteristic`参数, 可选, 用于设定所定义的存储过程的某些特征, 可包含内容及格式:

`[LANGUAGE SQL | [NOT] DETERMINISTIC | { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA} | SQL SECURITY { DEFINER | INVOKER} | COMMENT 'string']`

① `LANGUAGE SQL`指明编写存储过程的语言为SQL语言, 省略时默认为SQL语言。

② `[NOT] DETERMINISTIC`, `DETERMINISTIC`表示存储过程对同样的输入参数产生相同的结果, 表示是“确定的”; `NOT DETERMINISTIC`, 则表示会产生不确定的结果(默认)。

③ `CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA`, 四选一, 表示包含读或写数据的语句, 省略时表示 `CONTAINS SQL`。

`CONTAINS SQL`表示存储过程包含了SQL语句, 但不包含读或写数据的语句(如SET语句等)。

`NO SQL`表示存储过程不包含SQL语句。

`READS SQL DATA`表示存储过程包含SELECT查询语句, 但不包含更新语句。

`MODIFIES SQL DATA`表示存储过程包含更新数据的语句。

创建存储过程

说明:

④ **SQL SECURITY{ DEFINER | INVOKER}**子句用于指定存储过程执行时的权限验证方式，可指定为**DEFINER**或**INVOKER**，省略默认为DEFINER。

指定为DEFINER，MySQL将验证调用存储过程的用户是否具有存储过程的execute（执行）权限和DEFINER子句所指定的用户是否具有存储过程引用的相关对象的权限。

指定为INVOKER，MySQL将使用当前调用存储过程的用户执行此过程，并验证用户是否具有存储过程的execute权限和存储过程引用的相关对象的权限。

⑤ **COMMENT 'string'**子句用于给存储过程指定注释信息，其中string为描述内容，子句省略时，注释信息为空。

OPTION

创建存储过程

1. 在代码窗口中使用CREATE PROCEDURE语句创建存储过程

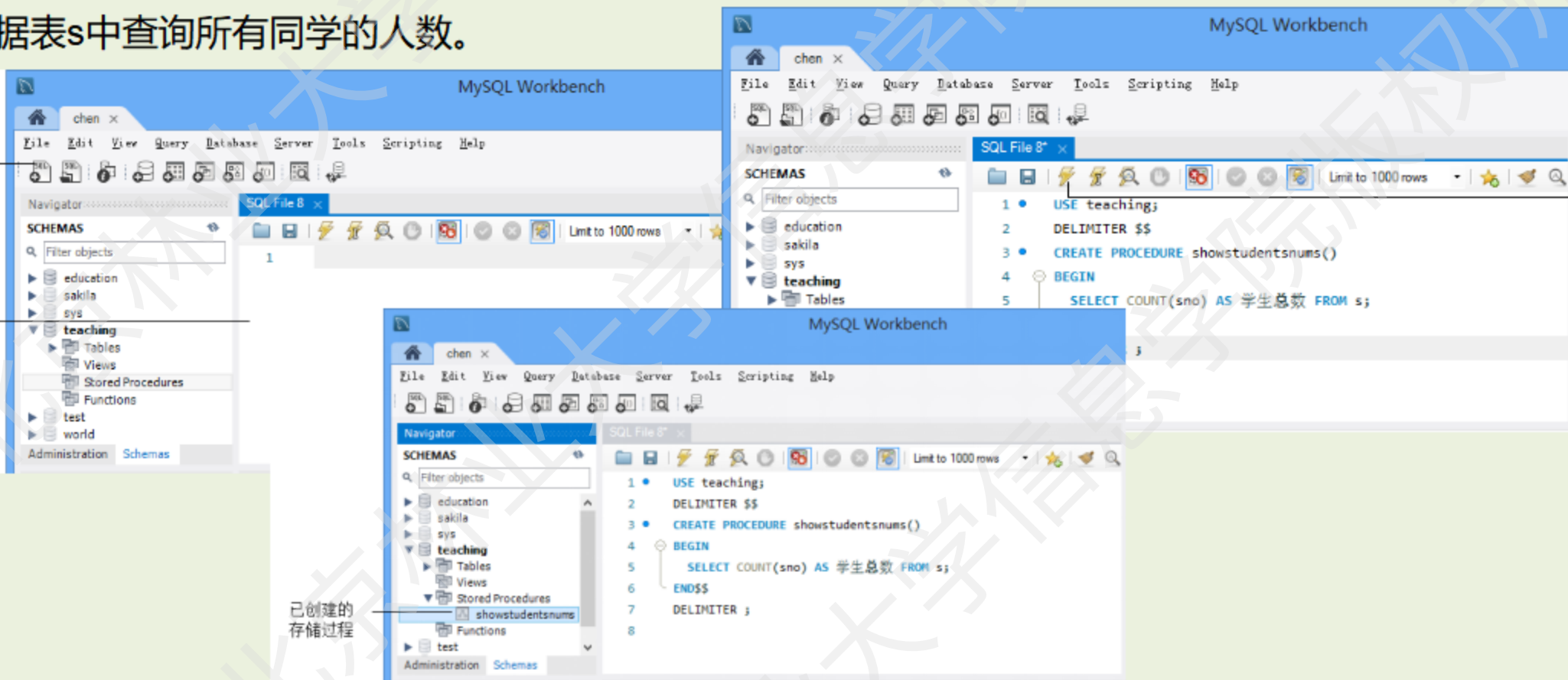
【例15-10】在teaching数据库中，创建一个名称为showstudentsnums的不带参数的存储过程，该存储过程的功能是从数据表s中查询所有同学的人数。

单击此按钮，新建一个SQL代码窗口

在此代码窗口中，输入SQL代码

已创建的存储过程

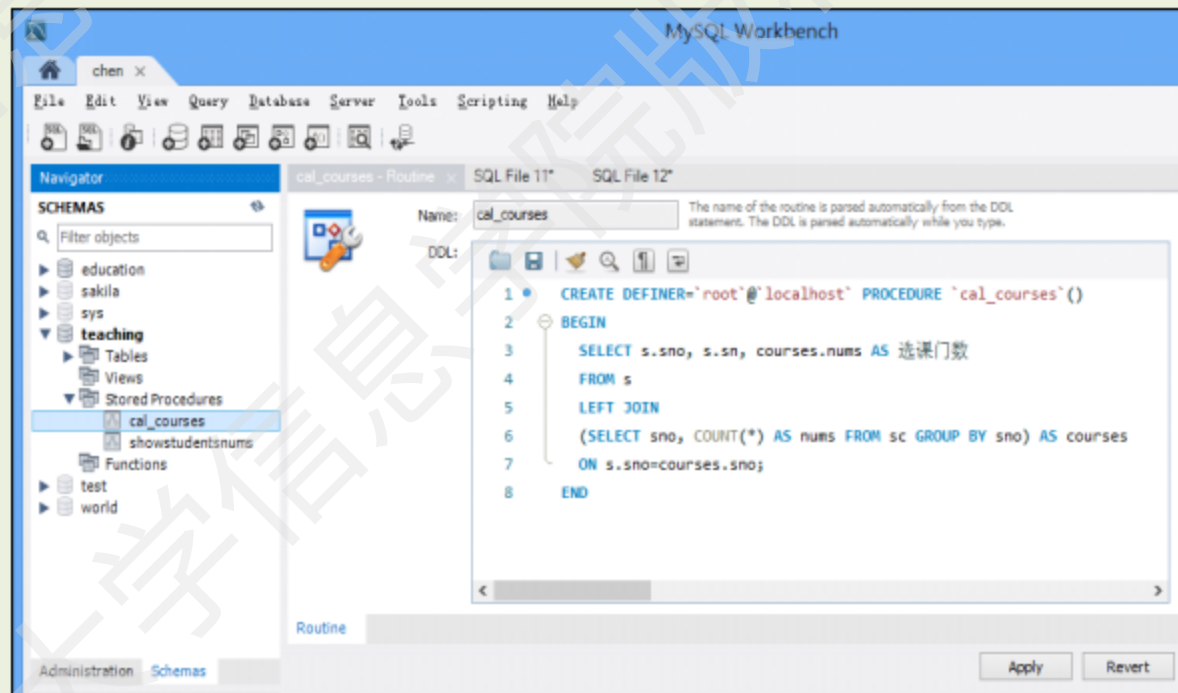
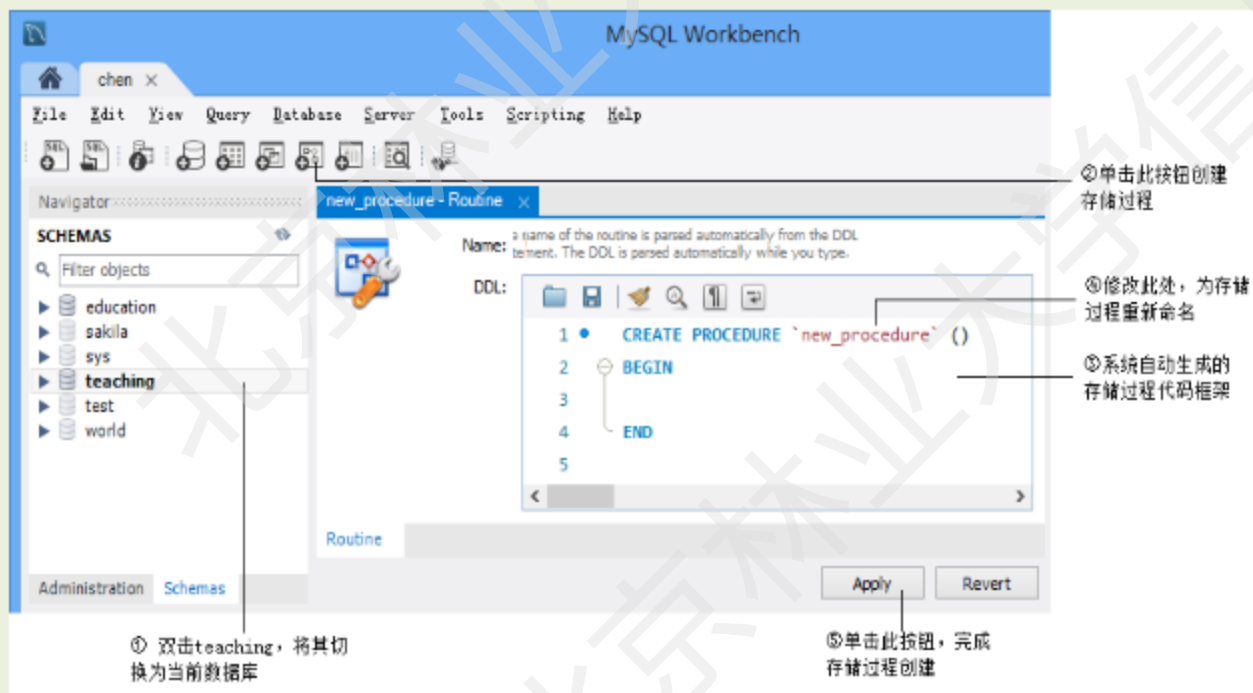
单击此按钮，执行代码，完成存储过程的创建



创建存储过程

2. 利用可视化管理工具MySQL Workbench功能菜单创建存储过程

【例15-11】在teaching数据库中，创建一个名称为cal_courses的不带参数的存储过程，该存储过程的功能是从学生表s和选课表sc中查询所有同学的选课数量（没有选课的同学，选课数量显示为NULL），列出学号、姓名和选课门数。



调用存储过程

OPTION

对于已创建好的存储过程，用户可以在MySQL中调用执行，可以在应用程序中调用。

此处，主要讲述在MySQL中调用执行，调用存储过程的语句是：

```
call procedure_name [(procedure_parameter)];
```

其中，

(1) `procedure_name`是已定义的存储过程的名称。

(2) `procedure_parameter`表示实际参数，即调用时应传入存储过程的参数。若不需要参数，则调用语句可简化为：`call procedure_name;` 或 `call procedure_name();`

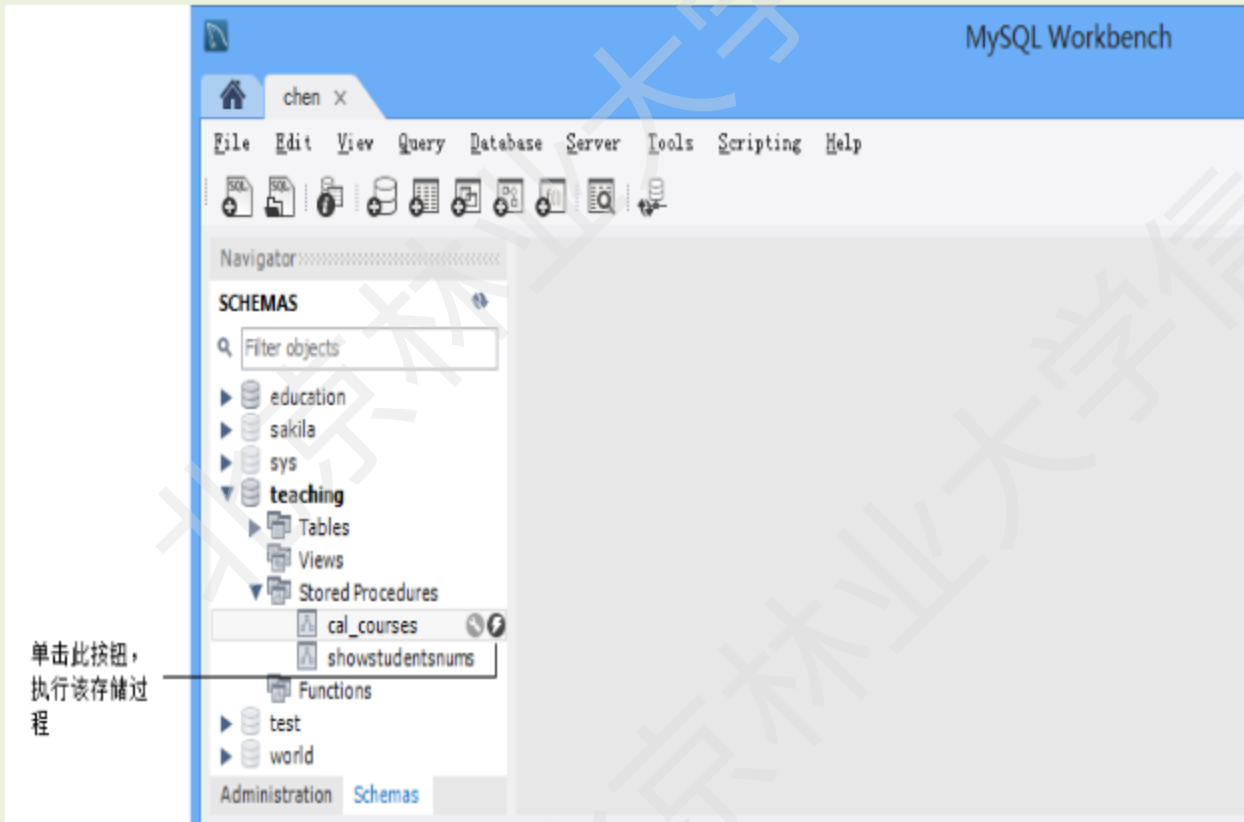
(3) 调用存储过程时，是否需要实际参数、需要几个实际参数，是由定义存储过程时的参数表的内容所决定。

调用存储过程

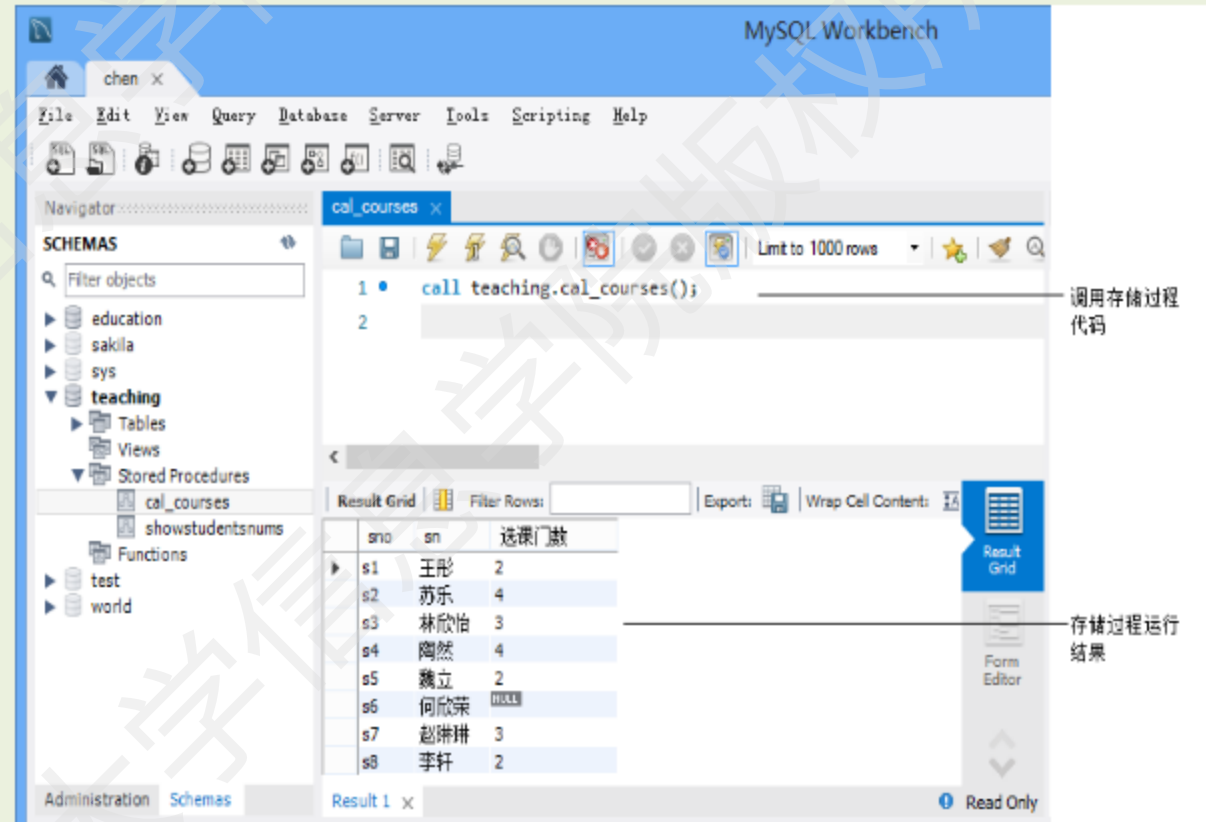
OPTION

以例15-11所定义的存储过程 “cal_courses”的调用为例，讲解在MySQL中调用执行存储过程的方法和步骤。

方法一：



方法二：



查看存储过程

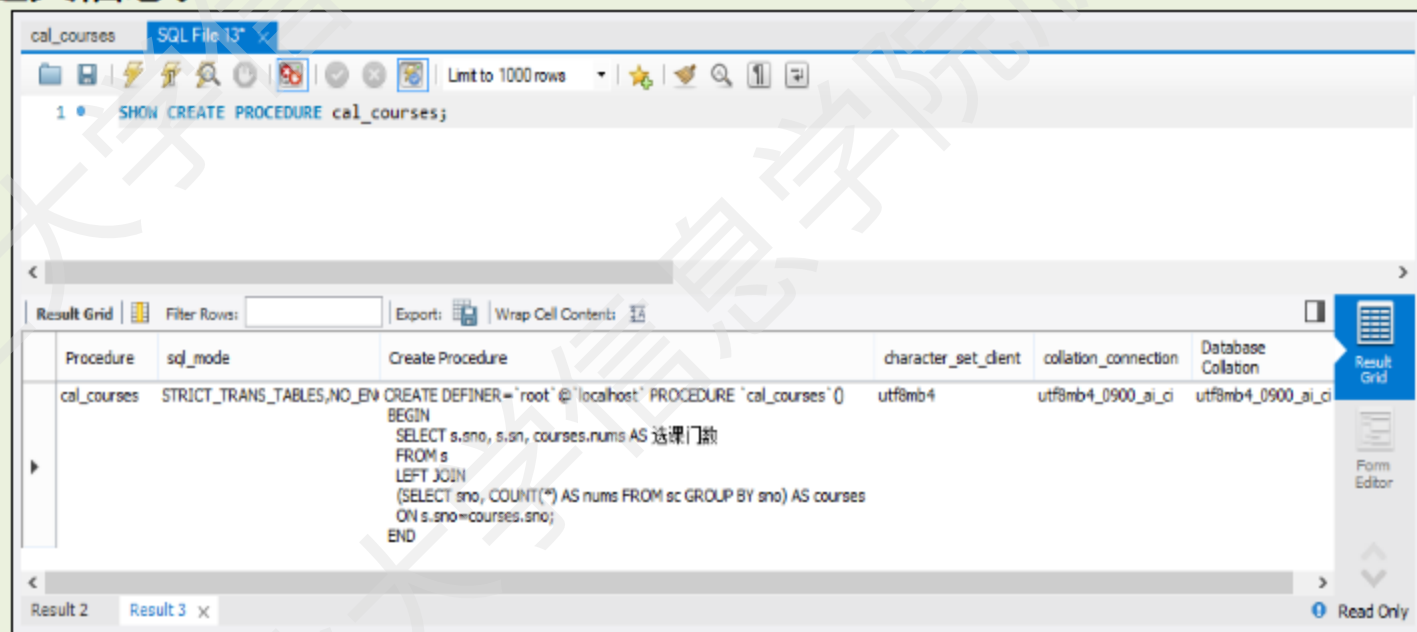
1. 使用SHOW CREATE PROCEDURE语句查看存储过程的定义

语句格式: **SHOW CREATE PROCEDURE** *procedure_name*;

*procedure_name*为存储过程的名称。

语句的作用: 查看存储过程的定义信息, 包括存储过程的名称、代码、字符集等信息。

比如: 查看存储过程 “cal_courses”的定义信息。



查看存储过程

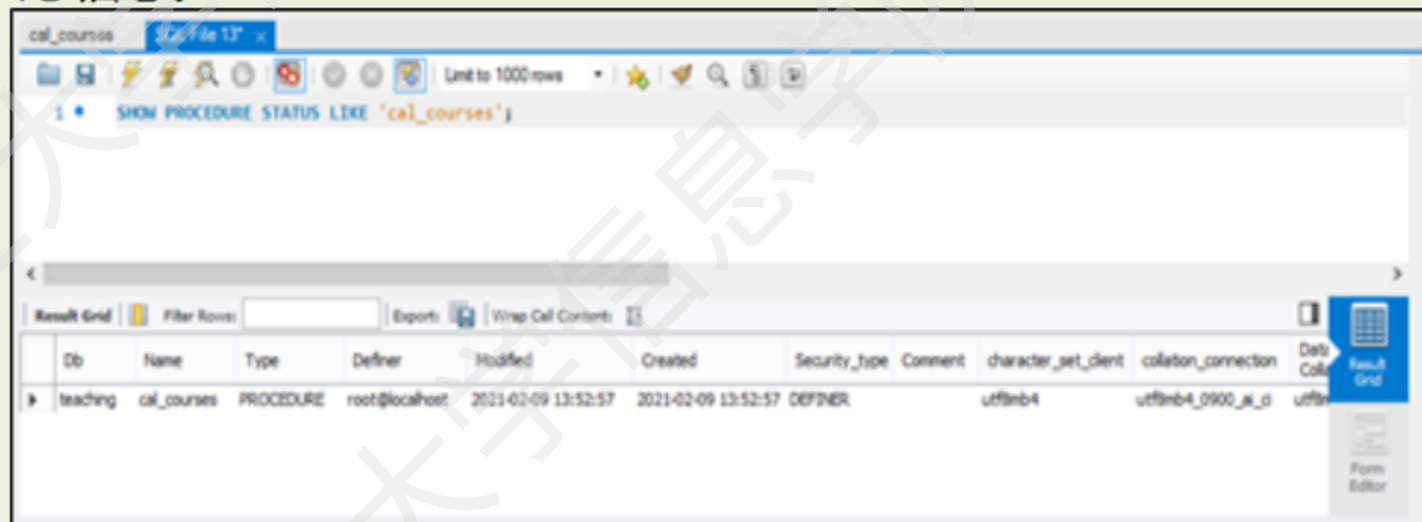
2. 使用SHOW PROCEDURE STATUS语句查看存储过程的状态特征

语句格式: **SHOW PROCEDURE STATUS LIKE 'pattern';**

pattern用来匹配存储过程名称的模式字符串, 其中可写普通字符, 也可写**%**和**_**通配符。

语句的作用: 查看名称与**pattern**所指定的模式相匹配的所有存储过程的状态特征信息, 包括: 所属数据库、存储过程名称、类型、定义者、注释、创建和修改时间、字符编码等信息。

比如: 查看存储过程 “cal_courses”的状态信息。



Db	Name	Type	Definer	Modified	Created	Security_type	Comment	character_set_client	collation_connection	Data Coll
teaching	cal_courses	PROCEDURE	root@localhost	2021-02-09 13:52:57	2021-02-09 13:52:57	DEFINER		utf8mb4	utf8mb4_0900_ai_ci	utf8mb4

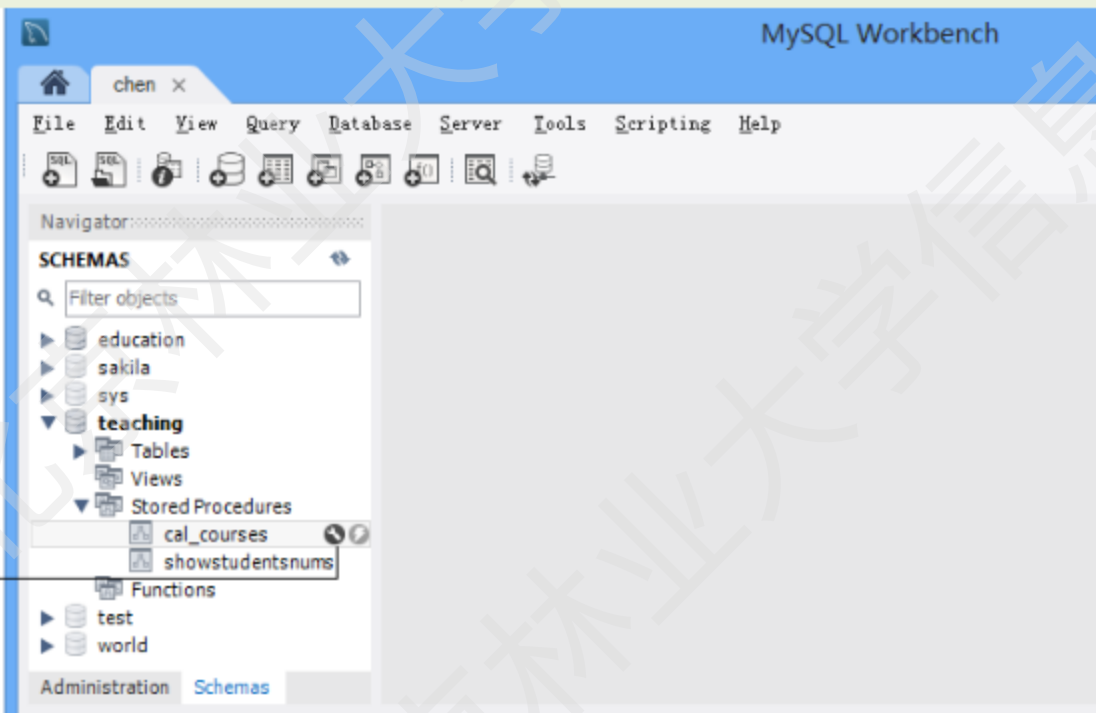
四

OPTION

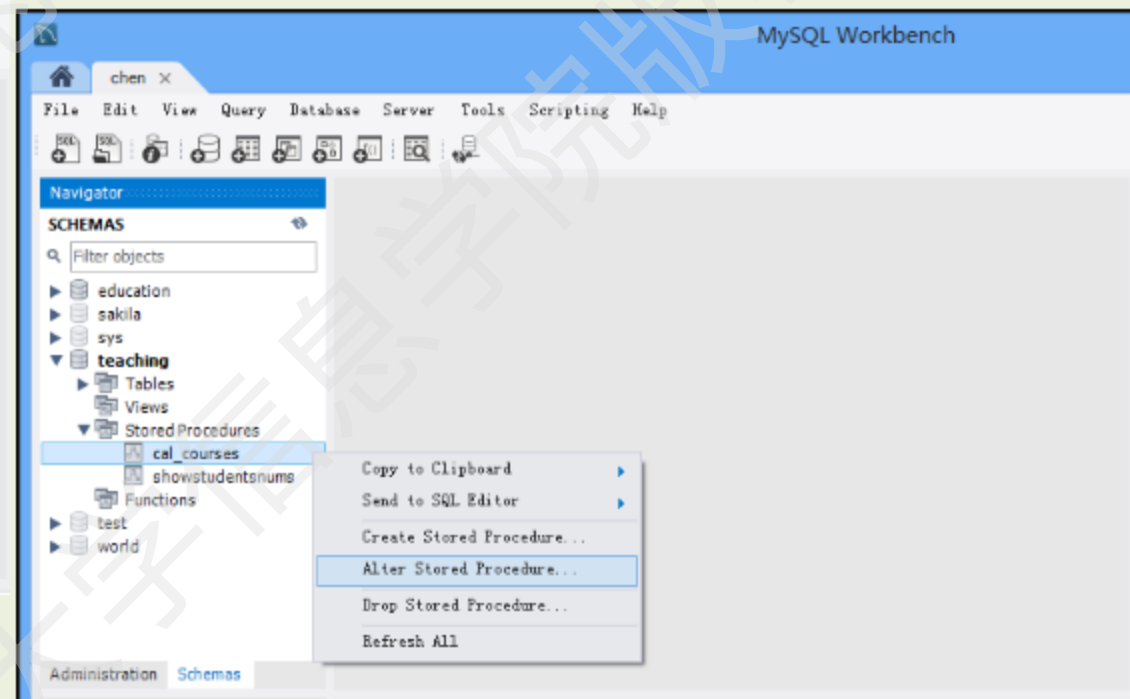
修改存储过程

1. 修改存储过程的内容代码

方法一：



方法二：





OPTION

修改存储过程

2. 修改存储过程的状态特征

通过ALTER PROCEDURE语句修改存储过程的状态特征信息，语句格式：

ALTER PROCEDURE *procedure_name* [**characteristic** ...];

其中，*procedure_name*表示已定义的存储过程；**characteristic**表示要更改的存储过程的特征信息（与CREATE PROCEDURE语句中有关说明一致）。

比如，修改cal_courses状态信息：

访问数据的权限改为MODIFIES SQL DATA

安全类型也变成了INVOKER

注释信息改为“显示学生选课门数”。

The screenshot shows a MySQL IDE window titled "SQL File '14'". It contains two SQL statements:

```
1 • ALTER PROCEDURE cal_courses MODIFIES SQL DATA SQL SECURITY INVOKER COMMENT '显示学生选课门数';
2 • SHOW PROCEDURE STATUS LIKE 'cal_courses';
```

Below the statements is a "Result Grid" showing the output of the second statement. The grid has columns: Db, Name, Type, Definer, Modified, Created, Security_type, Comment, and character_set_client. The data row shows:

Db	Name	Type	Definer	Modified	Created	Security_type	Comment	character_set_client
teaching	cal_courses	PROCEDURE	root@localhost	2021-02-09 15:02:12	2021-02-09 14:54:01	INVOKER	显示学生选课门数	utf8mb4

At the bottom of the screenshot, there is a caption: "存储过程的相关状态特征信息发生了改变" (The related status characteristic information of the stored procedure has changed).

五、
OPTION

删除存储过程

1. 利用DROP PROCEDURE语句删除存储过程

语句格式: **DROP PROCEDURE** [**IF EXISTS**] `procedure_name`

其中, `procedure_name`表示待删除存储过程名称;

IF EXISTS是可选的, 加上该关键字后, 系统先判断是否存在, 作用是防止因删除不存在的存储过程而引发错误。

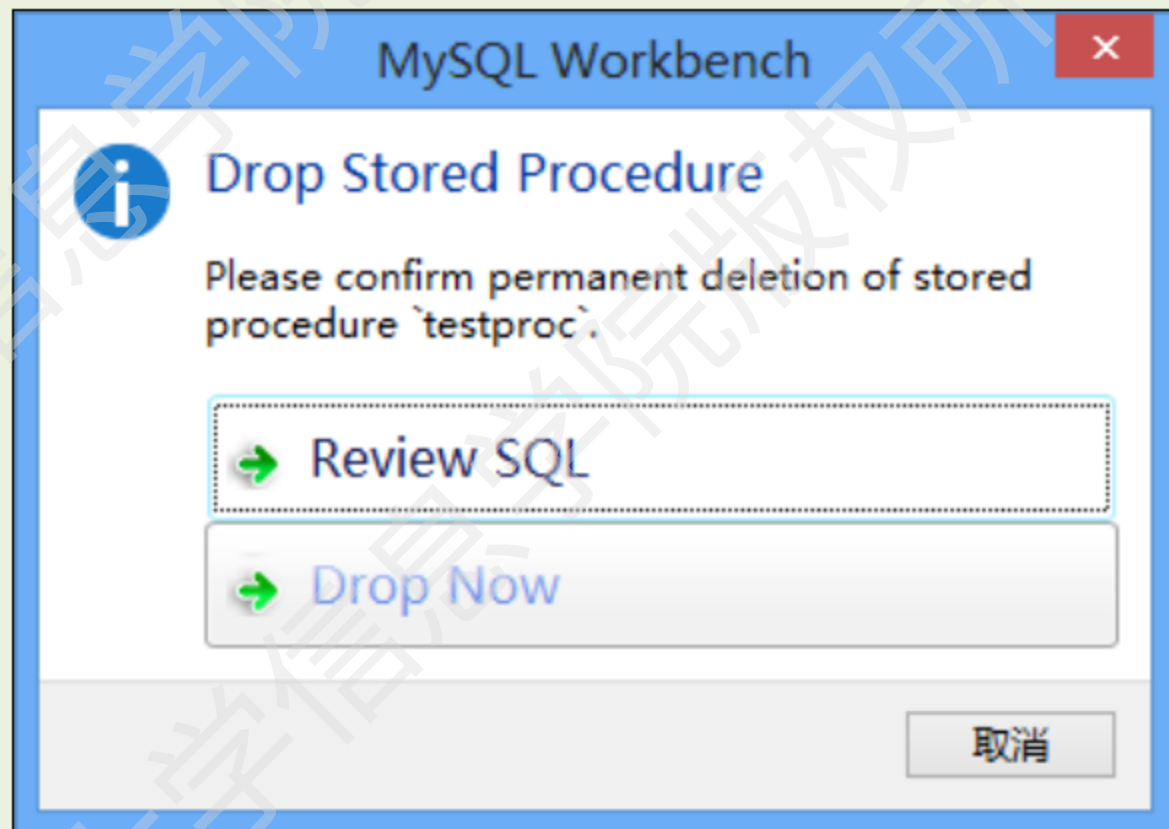
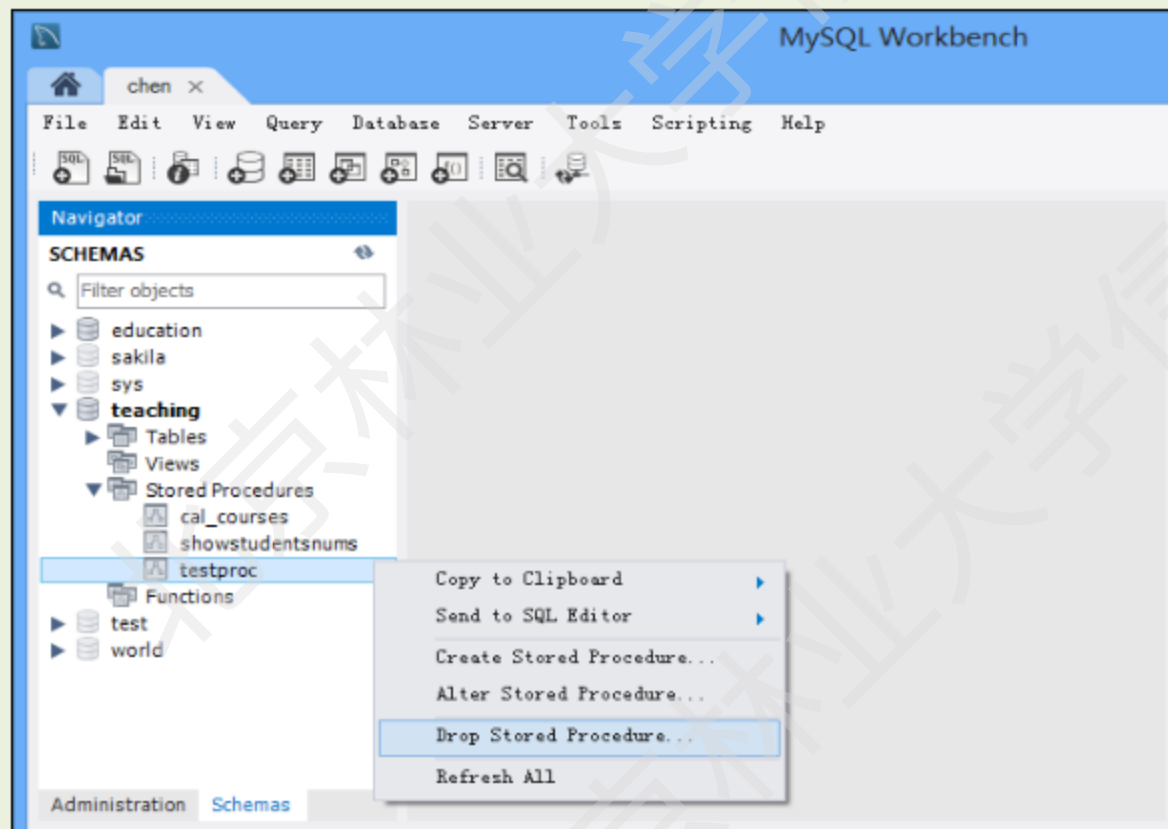
比如: 要删除存储过程testproc, 可以使用语句:

DROP PROCEDURE testproc;

五、
OPTION

删除存储过程

2. 利用可视化管理工具MySQL Workbench功能菜单删除存储过程





存储过程的参数

1. 形式参数

简称“形参”，是指在定义存储过程或函数时，写在存储过程名或函数名右侧括号内的变量名，它们用于接收从外界传递给该存储过程或函数的数据。

注意：

- (1) 形参指明了从调用过程处（即外界）传递给该存储过程的参数的个数和类型。
- (2) 形参是变量，形参表类似于变量声明，当还未发生过程调用时，形参无值，只有当发生过程调用时，形参才具有值，其值是由调用处的实参传递过来。



存储过程的参数

1. 形式参数

(3) 存储过程的参数可以有IN、OUT、INOUT三种传递类型，省略时默认为IN类型。

IN表示输入参数，要求在调用存储过程时，必须为该参数传入一个确定的值。

OUT表示输出参数，要求在调用存储过程时，必须为该参数传入一个用户会话变量（全局变量），用于将存储过程运算中的结果带出到调用处使用。该种参数的功能是将值从存储过程中带出。

INOUT表示输入输出参数，要求在调用存储过程时，必须为该参数传入一个有确定值的用户会话变量（全局变量），用于在存储过程运算中使用，同时，又可利用该参数将值从存储过程中带出。

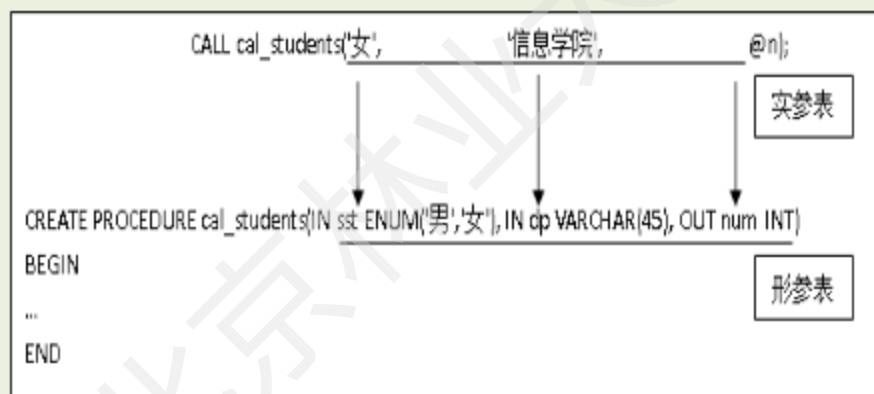


存储过程的参数

2. 实际参数

简称“实参”，是指在调用存储过程时，传送给被调存储过程的常量、变量或表达式。

实参个数、数据类型、参数传递类型必须与被调存储过程的形式参数的要求保持一致。



```
DELIMITER $$
CREATE PROCEDURE cal_students(IN sst ENUM('男','女'), IN dp VARCHAR(45), OUT num INT)
BEGIN
    DECLARE people INT DEFAULT 0;
    SELECT COUNT(*) INTO people
    FROM s
    WHERE sex=sst AND dept=dp;
    SET num=people; -- 将统计结果赋给参数num，通过该参数将值传出
END $$
```

- 执行调用语句CALL cal_students('女','信息学院',@n);时:
- 实参值'女'、'信息学院'分别传递给被调过程cal_students的形参sst和dp,
- 实参@n传递给形参num, 由于形参num是输出类型, 这样, 相当于形参num得到的是实参@n的地址, 所以, 在被调过程执行时, 对形参num的操作实质就是对实参@n的操作。在过程体中的语句SET num=people;, 相当于将计算结果存放到实参@n中。



存储过程的参数

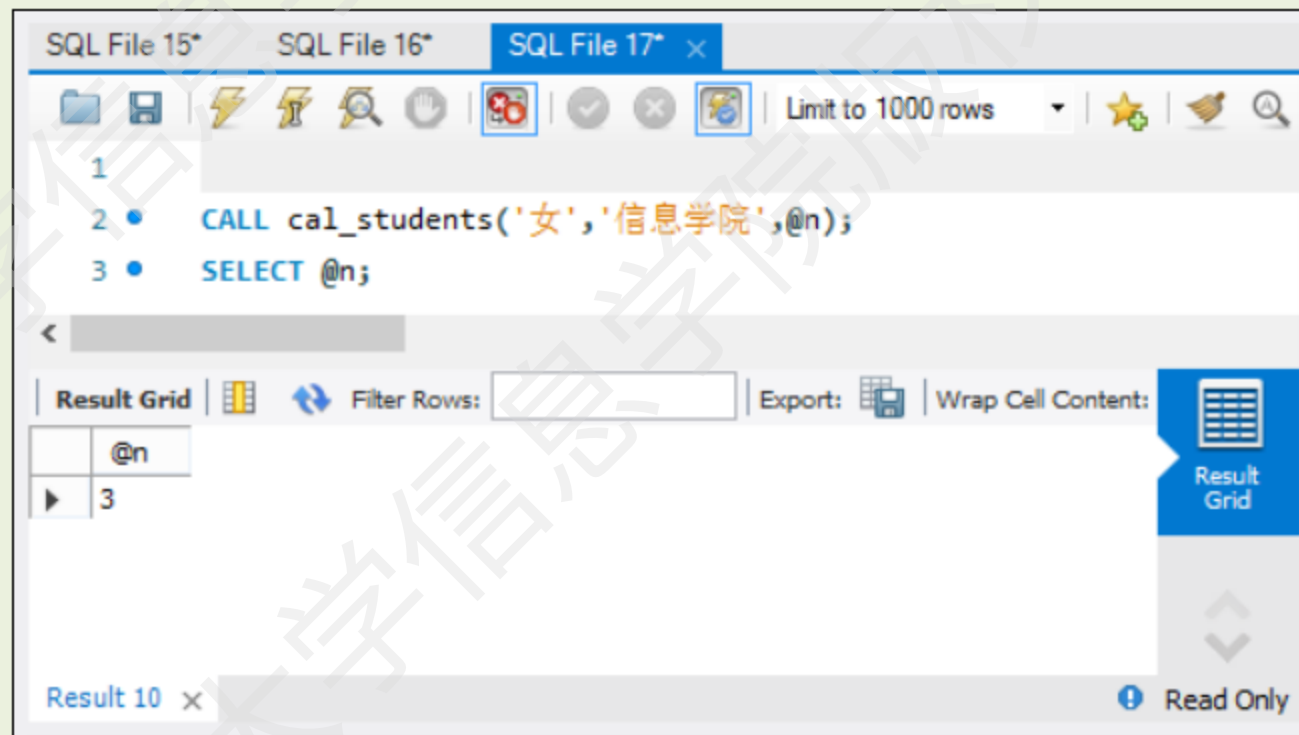
3. 调用有参存储过程

(1) 在代码窗口中直接输入CALL语句调用

比如，要调用存储过程cal_students，可在代码窗口中键入调用语句并执行：

```
CALL cal_students('女','信息学院',@n);
```

```
SELECT @n;
```



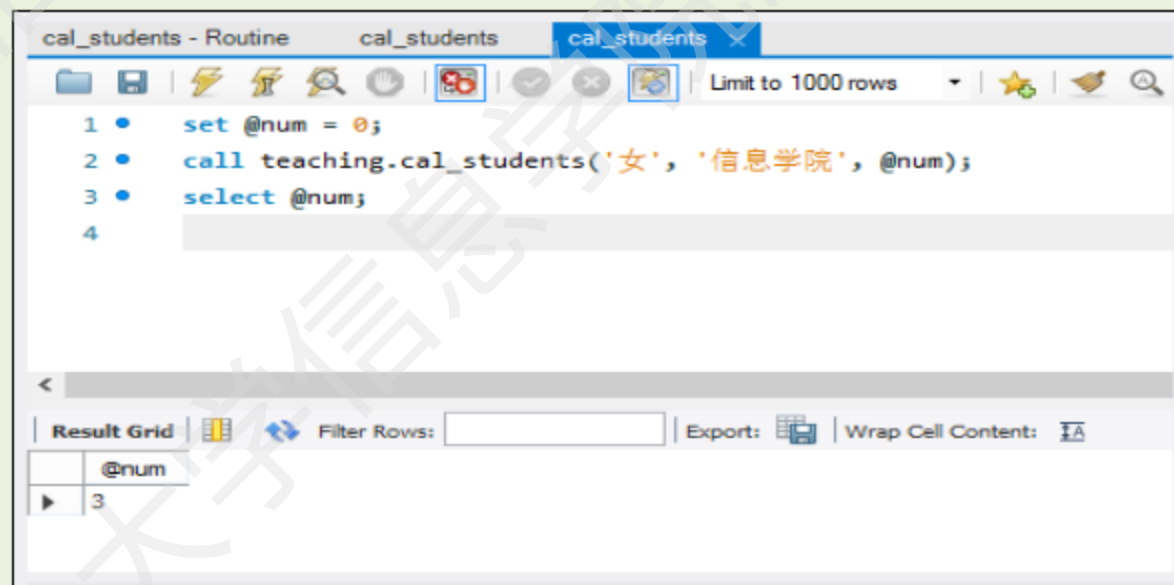
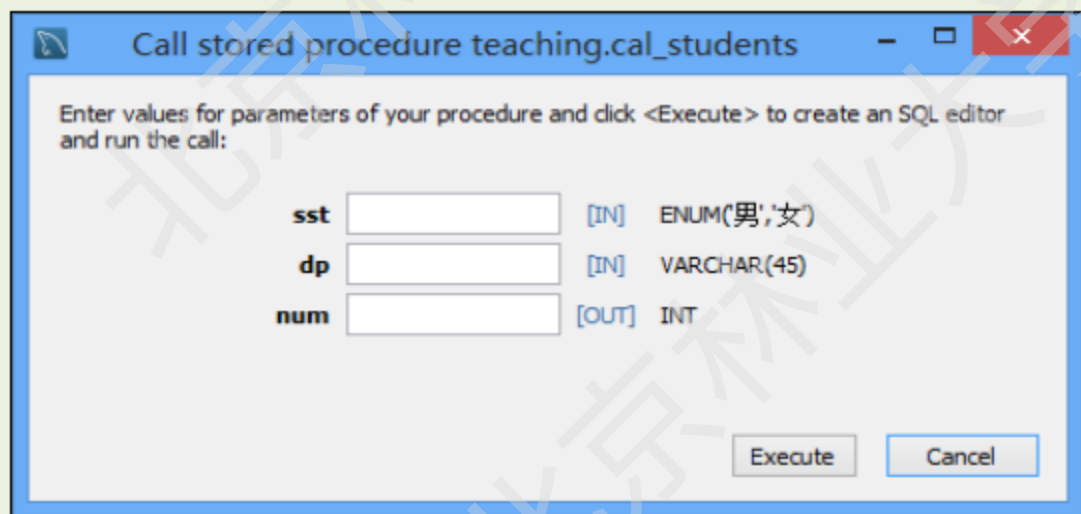


存储过程的参数

3. 调用有参存储过程

(2) 利用MySQL Workbench调用

比如，要调用存储过程cal_students，依次单击展开数据库“teaching”→“Stored Procedures”分支，可见在数据库“teaching”的“Stored Procedures”下已创建好的存储过程“cal_students”，用鼠标指向“cal_students”，出现运行存储过程图标，单击此图标出现如图对话框依次在三个文本框中输入：女、信息学院、@n，然后单击“Execute”按钮，执行。



15.5 MySQL用户自定义函数



创建函数

OPTION

1. 使用CREATE FUNCTION语句创建用户自定义函数

使用CREAE FUNCTION语句，语句格式：

```
CREATE [DEFINER = { user | current_user }] FUNCTION func_name([func_parameter[, ...]])
```

```
RETURNS type
```

```
[characteristic ...]
```

```
BEGIN
```

```
func_body
```

```
END
```

说明：

- (1) **func_name**是要创建的函数的名字，默认在当前数据库中创建函数。若需要在特定数据库中创建存储过程，则要在名称前面加上数据库的名称，即：数据库名.func_name。
- (2) **RETURNS type**指明函数返回值的数据类型，type表示数据类型。
- (3) **func_body**表示是函数体，由于函数运算结束后，必须有返回值，所以函数体中必须至少要有有一个RETURN语句，格式是：**RETURN value;**。
- (4) **func_parameter**是函数的参数（形式参数），函数的形式参数只能是IN类型，不能为OUT和INOUT类型。
- (5) **characteristic**用于定义函数的状态特征，它们的含义与格式同CREATE PROCEDURE语句中的规定。

创建函数

OPTION

1. 使用CREATE FUNCTION语句创建用户自定义函数

The screenshot displays the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the 'teaching' database selected, with the 'Functions' folder expanded, listing the 'f() isprime' function. On the right, the 'SQL File 4*' editor shows the SQL code for creating the 'isprime' function. The code includes a delimiter, the function definition with comments, and the execution command. Annotations with arrows point to specific UI elements: the 'New' button in the toolbar, the 'teaching' database in the Navigator, the 'isprime' function in the Functions list, and the 'Execute' button in the SQL editor toolbar.

```
1 DELIMITER $$
2 CREATE FUNCTION isprime( m INT)
3 RETURNS BOOL -- 函数返回值的数据类型
4 NO SQL -- 函数体不包含SQL语句
5 COMMENT '判断一个整数是否为素数, 若是, 返回true, 否则返回0'
6 BEGIN
7 DECLARE i INT DEFAULT 2;
8 IF m<=1 THEN
9 RETURN FALSE;
10 END IF;
11 WHILE i<=m-1 DO
12 IF m MOD i=0 THEN
13 RETURN FALSE;
14 END IF;
15 SET i=i+1;
16 END WHILE;
17 RETURN TRUE;
18 END $$
19 DELIMITER ;
```

①单击此按钮, 新建代码窗口

当前数据库

④已定义的自定义函数 isprime

②单击此按钮, 运行代码, 创建函数

③在代码窗口中, 输入创建自定义函数的代码

创建函数

OPTION

2. 利用可视化管理工具MySQL Workbench功能菜单创建自定义函数

MySQL Workbench

chen x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

- education
- sakila
- sys
- teaching
 - Tables
 - Views
 - Stored Procedures
 - Functions
- test
- world

new_function - Routine

Name: The name of the routine is parsed automatically from the DDL itemment. The DDL is parsed automatically while you type.

DDL:

```
1 CREATE FUNCTION `new_function`()
2 RETURNS INTEGER
3 BEGIN
4
5 RETURN 1;
6 END
7
```

Routine

Administration Schemas

Apply Revert

单击此按钮，创建自定义函数

系统自动生成的创建函数的框架代码

调用自定义函数

OPTION

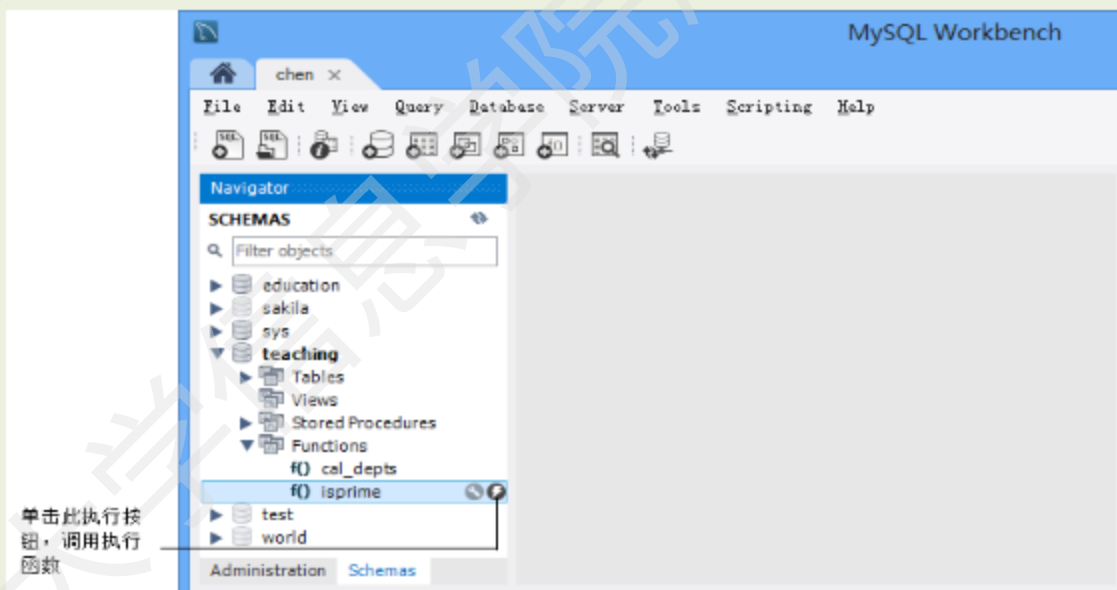
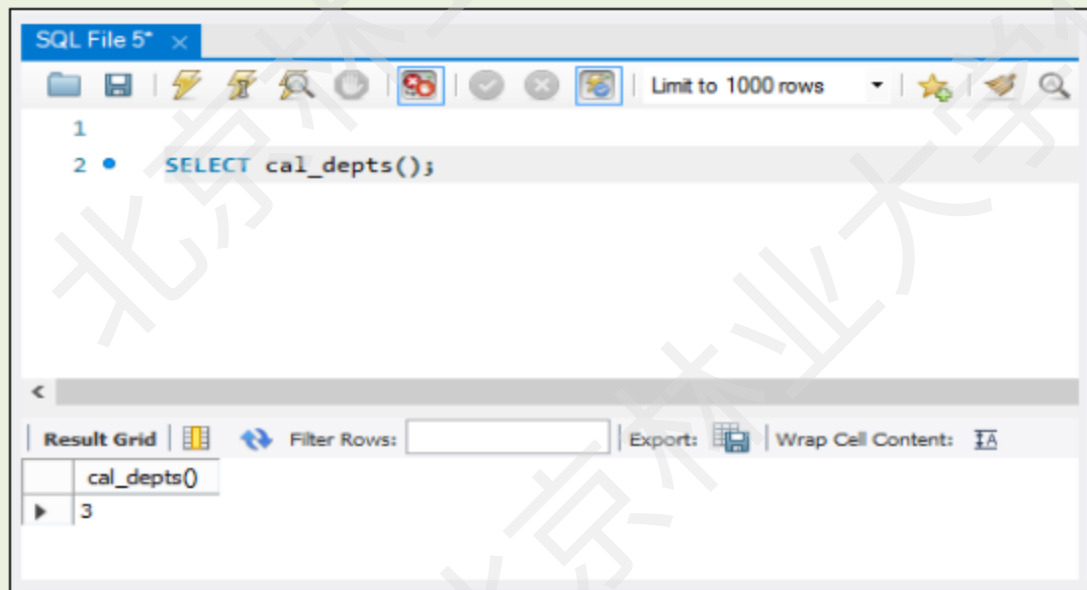
自定义函数的调用方法与MySQL内置函数的调用方法是一样的，调用方式：**函数名(参数)**

在MySQL Workbench中，可使用以下两种方法调用函数：

①**方法1**：在代码窗口中，直接输入调用函数的语句并执行。

比如，调用已创建的存储函数cal_depts()，在代码窗口中，输入并执行语句SELECT cal_depts();

②**方法2**：使用MySQL Workbench的函数调用执行按钮





函数的维护管理

1. 查看函数

(1) 查看函数的状态特征信息

①查看所有的自定义函数的状态特征信息，语句格式：

SHOW FUNCTION STATUS

②查看函数名与某一字符串模式匹配的所有自定义函数的状态特征信息，语句格式：

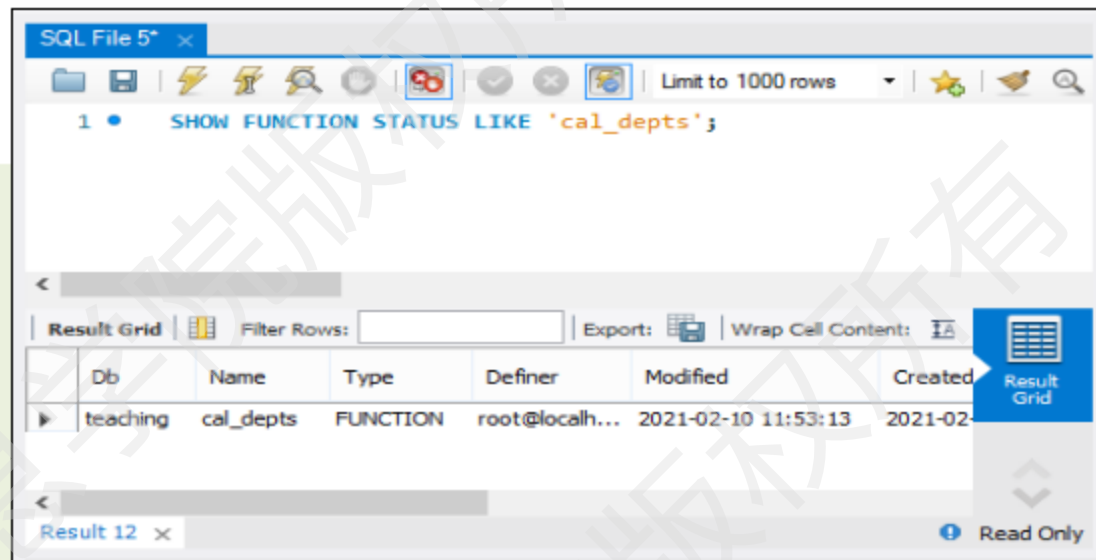
SHOW FUNCTION STATUS LIKE 'pattern';

其中，pattern用来匹配函数名称的模式字符串，其中可以写普通的字符，也可以写%和_通配符。

(2) 查看函数的代码

使用SHOW CREATE FUNCTION语句查看函数的代码，格式如下：

SHOW CREATE FUNCTION func_name;





函数的维护管理

2. 修改函数

(1) 修改函数的状态特征信息

使用ALTER FUNCTION语句，格式如下：

ALTER FUNCTION func_name [characteristic ...]

其中，**characteristic**表示要更改的函数的特征信息，与CREATE PROCEDURE语句中有关的说明相同。

比如，在代码窗口中，输入并执行如下语句，可将函数isprime的安全类型修改为INVOKER。

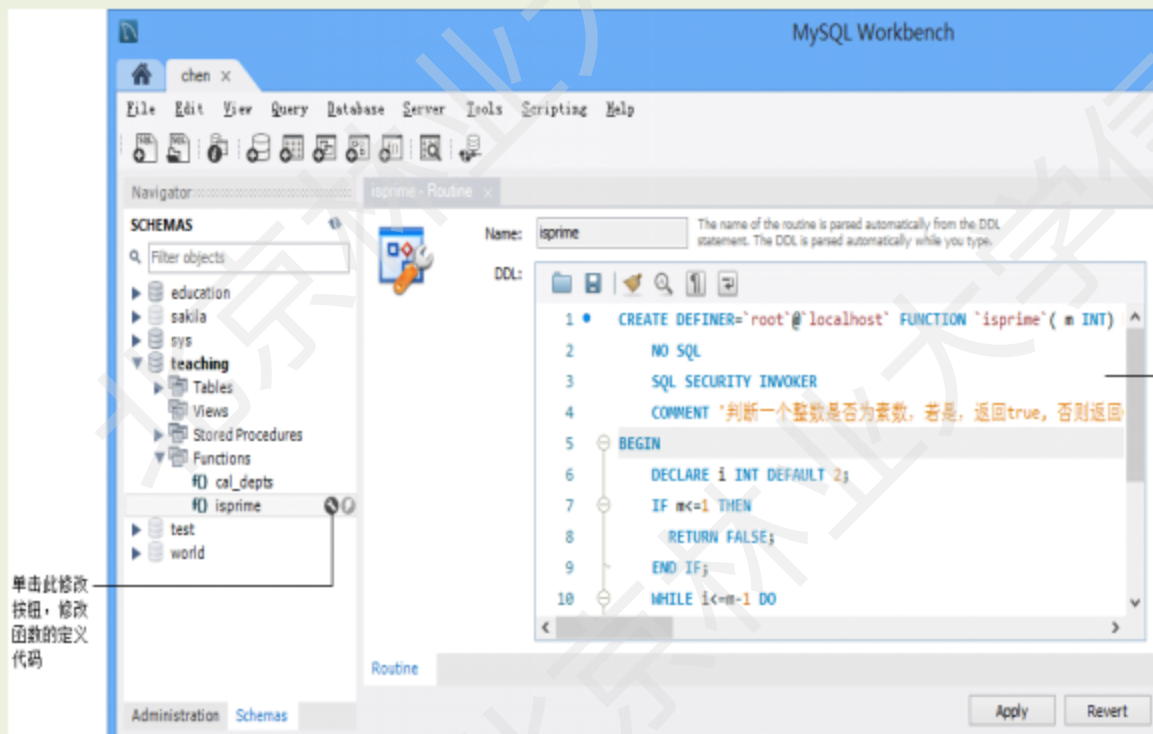
ALTER FUNCTION isprime SQL SECURITY INVOKER;

函数的维护管理

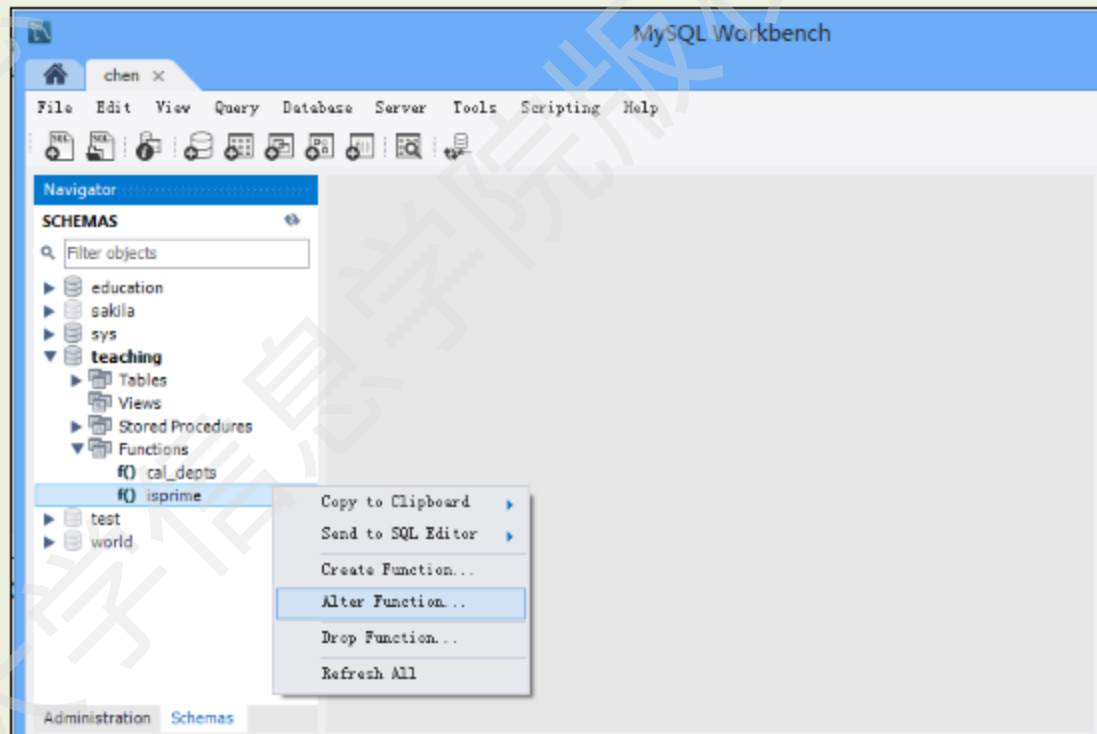
2. 修改函数

(2) 修改函数的代码

在MySQL Workbench中，修改函数的代码两种方法：



代码窗口中显示被修改函数的定义代码





函数的维护管理

3. 删除函数

删除函数，使用DROP FUNCTION语句，格式如下：

DROP FUNCTION func_name;

比如，要删除函数“cal_depts”，可在代码窗口中输入并执行如下语句：

DROP FUNCTION isprime;

15.6 游标的使用

1、

OPTION

游标的定义

2、

OPTION

打开游标

3、

OPTION

从结果集中提取数据

4、

OPTION

关闭游标

1、 OPTION

游标的定义

游标的概念：游标实际上是一种能从包括多条数据记录的结果集中逐条访问这些记录的机制。

- (1) 游标主要包括结果集和游标位置两部分，游标结果集是定义游标的SELECT语句的结果集，游标位置（游标指针）则是指向这个结果集中的某一行的指针。
- (2) 利用游标，可以对结果集中的每一条记录顺序地从前向后逐条进行遍历。
- (3) MySQL游标只能用于存储过程和函数。
- (4) 游标的使用过程和顺序，包括声明游标、打开游标、从结果集中提取数据、关闭游标。

1、

OPTION

游标的定义

使用DECLARE语句声明游标，语法格式：

```
DECLARE cursor_name CURSOR FOR select_statement;
```

其中，

cursor_name表示游标的名称；

select_statement为一SELECT语句，用于生成游标操作的结果集。

比如，在teaching数据库中为学生表s创建一个游标，名称为s_cursor，对应结果集为学生表中的学生学号和姓名，语句：DECLARE s_cursor CURSOR FOR SELECT sno,sn FROM s;

声明游标后，与该游标相对应的SELECT语句并没有执行，服务器内存中还不存在结果集。

2、 OPTION

打开游标

游标使用之前必须先打开，打开游标使用OPEN语句，语法格式：

OPEN cursor_name;

比如，要打开创建的游标s_cursor，使用如下语句：OPEN s_cursor;

打开游标后，与该游标声明相对应的SELECT子句将被执行，服务器内存中将存放对应的结果集，且游标指针指向结果集中的第一条记录。

3、 OPTION

从结果集中提取数据

打开游标后，可使用FETCH语句从结果集中提取数据，每次只能取出游标当前指针指向的一条记录，FETCH语句每取出一条记录，游标指针自动向后移动并指向下一条记录。

FETCH语句的语法格式：

```
FETCH cursor_name INTO var_name1 [, var_name2 ]....;
```

4

OPTION

关闭游标

游标使用完毕后，要用CLOSE语句关闭，语法格式：

```
CLOSE cursor_name;
```

关闭游标的目的是释放游标打开时产生的结果集，以通知服务器释放游标所占用的资源，节省MySQL服务器的内存空间。

比如，要关闭已创建的游标s_cursor，可使用语句：CLOSE s_cursor;



人邮教育

www.ryjiaoyu.com

更多样书申请和资源下载需求，请登录人邮教育社区
(www.ryjiaoyu.com)

海量图书方便查询

囊括各大品类，您想要的应有尽有

免费申请样书

教师免费申请样书，
我们将安排快递迅速送达

下载配套资源

教学视频、PPT课件、教学案例、
习题答案、模拟试卷等丰富资源
免费下载



优惠购书

教师可以申请最低折扣
学生直接优惠购买图书

成为作者

欢迎写文章 / 投稿，我们强大的编辑团队将为您提供专业和高效的编辑出版服务